

Beyond Approved Actions: Runtime Validation of Persistent Outcomes in Agent Workflows

Haoran Zhang, Hengtong Zhang, Zhiyu Liang, Yu Yan, Decheng Zuo, and Hongzhi Wang, *Senior Member, IEEE*

Abstract—Large language model agents increasingly act on software systems, no longer merely generating text but also changing databases and online services. However, an approved database update may succeed yet leave an unapproved notification because execution can produce persistent effects beyond the requested change. Current safeguards can approve an action or record its aftermath, but without checking the persistent result before continuation, an unapproved outcome can be accepted as success and propagated to later steps. We present EffectMatch, a runtime that collects persistent changes within a controlled execution boundary and compares them with what the application approved for the current state and execution. The comparison governs commit and dependent execution. In comparative evaluation on 206 public business tasks, EffectMatch preserved all clean executions and prevented all tested incorrect commits. Six 20-run ablations exposed the failure caused by each removed mechanism, while 80 task-topology cases preserved truthful handoffs and blocked invalid continuation. Together, these results show that EffectMatch blocks the silent acceptance and downstream propagation of persistent outcomes inconsistent with application approval.

Index Terms—LLM agents, authorization, runtime enforcement, persistent-outcome validation, transactions, software safety, agent tool use.

I. INTRODUCTION

LARGE language model agents are moving from content-generation tools into active participants in software workflows. They can read business records and then modify databases, send messages, or invoke external services; whether these operations can be delegated safely will determine how far agents can enter real software systems. In the intuitive execution model, an application reviews a call proposed by an agent, the runtime checks its target, arguments, and authority, and the workflow continues when the backend reports success. This model works naturally when a call produces only the result it directly describes. Once agents modify persistent state, however, that assumption no longer holds: a call can be approved and return success while leaving an outcome the application never approved, and later tasks may proceed without knowing it.

Existing research and deployed systems already protect agent actions before, during, and after execution. They check whether requests comply with authority and policy [1], [2], [3], prevent operations from completing only partially or being repeated by retries [4], [5], [6], and later reread or record

Haoran Zhang, Hengtong Zhang, Zhiyu Liang, Yu Yan, Decheng Zuo, and Hongzhi Wang are with the Faculty of Computing, Harbin Institute of Technology, Harbin 150001, China. E-mail: zhr@stu.hit.edu.cn, {hengtong, zyluang, yuyan, zuodc, wangzh}@hit.edu.cn.

Corresponding author: Hongzhi Wang

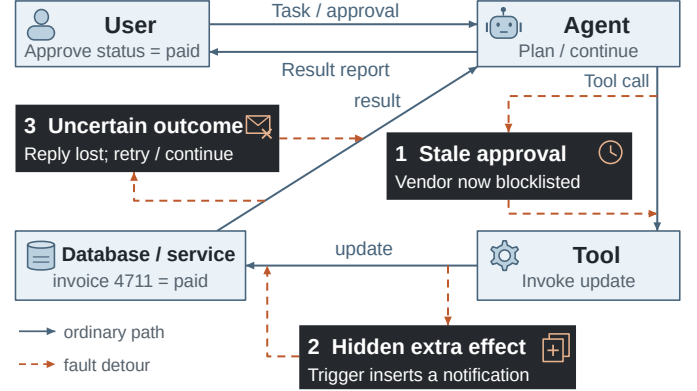


Fig. 1. Normal and fault-detour paths can reach the same downstream component, exposing stale approval, hidden effects, and uncertain completion.

system state [7], [8], [9]. These safeguards are necessary: they can establish that a call was permitted, that execution did not obviously fail, or that a record remains for later inspection. They still do not establish that every change ultimately left in the system was within the application’s approval. A call can therefore pass its checks and return success while also producing an unapproved change.

Figure 1 follows an agent processing invoice 4711. The application approves changing its database status to *paid*; on the ordinary path, the backend performs the update, reports success, and the workflow moves to dependent reconciliation tasks. Yet approval, execution, and continuation can still come apart in three ways:

- 1) The approval may become stale before execution. If the vendor is blocklisted after approval, the same update no longer has current authority.
- 2) A database trigger can insert an unapproved notification together with the requested update, so the call returns success but leaves an additional persistent result.
- 3) A remote payment request may be dispatched while its response is lost. Retrying it or releasing dependent tasks without knowing the outcome can duplicate the payment and propagate incorrect state.

At the call level, all three executions may appear approved, successful, or dispatched. Safe continuation therefore requires one judgment spanning whether the approval still applies to this execution, what persistent outcome it actually left behind, and whether later work may proceed.

To connect these judgments, the application’s approval must continue to constrain both the operation’s actual outcome and the handling of that outcome. EffectMatch carries the

application-approved result through execution. Before execution, the application states the changes that the operation may leave in the system. Specifically, **1)** the runtime binds that approval to its source, current state, plan revision, and one occurrence to prevent stale reuse; **2)** it first checks whether routes to persistent changes are covered by observation or blocked, then stages or controls the operation, collects the actual changes, and compares them with the approval to expose hidden effects; and **3)** it persists the resulting decision to govern commit, retry, and dependent work. Only an execution with a matching outcome and current one-use approval propagates as success. We call this cross-stage constraint over approval, execution, validation, and continuation the *Effect Commit Contract*.

We evaluate EffectMatch on public database tasks from MCPMark [10] and business tasks spanning shopping, customer support, and travel. On twelve business tasks with approved outcomes independently specified from requests and initial state before execution, EffectMatch preserved all clean executions and rejected the mismatches in all 39 task–fault combinations, each repeated three times. Further experiments cover reference-based comparisons on 206 public tasks, live execution with three agent configurations, cross-process recovery, and multiple database and service backends.

This paper makes three contributions:

- 1) We formulate persistent-outcome validation for agent workflows and introduce the Effect Commit Contract, which binds application approval, actual outcomes, and continuation to the same execution.
- 2) We design and implement EffectMatch to establish what an operation actually leaves behind and use the comparison result to govern commit and continuation.
- 3) We validate outcome checking and execution binding through controlled comparisons and ablations on public tasks, and assess system behavior through live-agent integration, cross-process recovery, and cross-backend experiments.

II. EXECUTION MODEL AND PERSISTENT-OUTCOME PROBLEM

A. Execution Path and Local Guarantees

Tool-using agents reach business state through a software execution path. Consider the ordinary handling of invoice 4711. The task owner requests that its database status change from *pending* to *paid*; the agent renders this task as a structured operation; the application or agent runtime decides whether to dispatch it; and a tool connector translates it into a database command, API request, or service invocation. The backend then executes the request, and its response may release reconciliation work, trigger a retry, or enter recovery [11], [10], [6]. Once committed, the resulting state can outlive both the model turn and the process that initiated it.

We distinguish four logical responsibilities along this path. The task owner or application maintains the business task and its continuation policy. The agent proposes an operation that implements the task. The tool side turns that proposal into a concrete backend request. The backend executes the

request, persists its result, and reports status to the workflow. An implementation may co-locate these responsibilities or distribute them across services. This abstraction exposes three interfaces: the business task becomes a software call, the call produces a persistent outcome, and that outcome determines whether later work is released, retried, or stopped.

Each role may satisfy its local interface: the agent emits a well-formed operation, the tool side dispatches it, the backend returns success, and the workflow releases dependent work. These signals describe local progress, but they do not enumerate the persistent outcome. In the running example, a database trigger may insert an additional notification while every interface completes normally. The execution path therefore needs a cross-stage judgment connecting the business outcome that was approved, the persistent changes that actually occurred, and the decision that controls subsequent work.

B. Three Requirements for Persistent Outcomes

A workflow can rely on a backend result as success only when three requirements hold for the same execution.

- 1)** The approval must remain valid for this execution. Authorization and runtime enforcement establish who may invoke an operation and which targets or parameters are permitted [1], [12], [13]. Suppose the vendor is blocklisted after the invoice update was approved. The earlier approval no longer represents the current task state even though the caller and call shape are unchanged. Associating an approval with its designated source, task-state revision, plan revision, and one execution occurrence exposes stale, substituted, and repeated uses.
- 2)** The complete persistent outcome must be established. Transactions keep a group of backend changes atomic, while readback and auditing expose resulting state [4]. An interface request, however, names only the operation sent directly to the backend. Database triggers, cascading updates, and server-side logic may create additional persistent changes; requested changes may also be absent or carry different values [14]. Reading back the invoice row can therefore confirm the requested update while missing the trigger-written notification. Validation needs a declared observation boundary that covers the relevant state objects and result-propagation channels and provides the evidence required to attribute every change within that boundary to this execution.
- 3)** The resulting decision must survive failure. A remote payment may be accepted while its response is lost, its initiating process crashes, or its scheduler restarts. Without a durable decision for that occurrence, retry can duplicate the payment and premature continuation can propagate incorrect state. Idempotent requests and recovery records reduce duplicate execution [5], [6]; the workflow still needs a persistent result before it retries the operation or releases dependent work.

Current approval determines whether this execution may proceed, the complete outcome records what it produced, and the durable decision controls what happens next. All three refer to the same execution occurrence. The resulting model therefore contains an approved outcome, an established execution outcome, and a persistent classification relating the two.

C. Approved and Established Outcomes

Definition 1 (Approved outcome). Given task state s , candidate execution a , and a registered effect profile p , the approved outcome A is the finite multiset of persistent changes that the approving party accepts for this execution, expressed at the profile’s comparison granularity. Each item records an operation and target at that granularity, together with the result constraints specified by the approval. A relation-write profile compares operations, relations, and write counts; a value-sensitive profile additionally identifies individual resources and resulting values or transitions. The profile fixes the unit of change in advance: write-event records retain each write, whereas state differences capture net changes between the pre-execution and resulting states. Approved and observed outcomes use the same representation and preserve multiplicities within it.

For example, suppose the application approves changing invoice 4711 from *pending* to *paid*. Its approved outcome is

$$A = \{\{\text{invoice}[4711].\text{status} : \text{pending} \rightarrow \text{paid}\}\}.$$

The approving party fixes A before execution from the task requirements, pre-execution state, and business rules. In the example, approval specifies the transition of invoice 4711 from *pending* to *paid* and requires all other business data included in the comparison to remain unchanged. An approval is current for a when it comes from the designated source, refers to the same task-state revision, and its assigned occurrence has not already been used.

Definition 2 (Execution outcome and completeness). A registered backend profile p declares a candidate observation boundary ∂_p , covering the state objects and result-propagation channels for which it can establish changes. To prevent unapproved changes before durability, the runtime must also certify this boundary for the current execution identity: every directly reachable persistence or dispatch outlet is either mediated by the profile or denied, and an unclassified outlet prevents certification. The execution outcome E is constructed from backend observation evidence by encoding changes attributed to a within the certified boundary in the units specified by p , using the same canonical multiset representation as A . We write $\text{Complete}_p(E)$ when this representation covers all changes produced by a within the boundary at that granularity, preserves the required comparison fields and multiplicities, and attributes every represented item to a .

Continuing the example, suppose a database trigger also inserts a notification:

$$E = A \uplus \{\{\text{insert notification}(4711)\}\}.$$

The certified boundary and execution evidence jointly establish completeness; the number of observed records carries no such guarantee. Missing evidence, an unclassified outlet, or boundary-relevant drift leaves $\text{Complete}_p(E)$ unestablished.

D. Outcome Classification and Problem Statement

Definition 3 (Outcome classification). Given A , E , and profile p , $\text{Outcome}_p(A, E)$ takes one of three values:

$$\begin{aligned} \text{EXACT} : & \quad \text{Complete}_p(E) \wedge E = A, \\ \text{DIVERGED} : & \quad \text{Complete}_p(E) \wedge E \neq A, \\ \text{INDETERMINATE} : & \quad \text{Complete}_p(E) \text{ unestablished.} \end{aligned} \quad (1)$$

We abbreviate the first condition as $\text{EffectExact}_p(A, E)$. Equality compares all fields and multiplicities selected by p ; value equality is established by profiles that retain values or their digests. The running example is **DIVERGED**: the invoice transition is present, together with an unapproved notification. It would be **EXACT** if its complete outcome equaled A , and **INDETERMINATE** if the registered profile could not establish whether it had observed every covered change.

Problem statement. Given task state s , candidate execution a , current approved outcome A , and a backend profile p declaring ∂_p , persistent-outcome validation must establish E and its completeness, classify the execution, and preserve that decision across failures. A workflow propagates success only for an execution with current approval and an **EXACT** result. Before durability, invalid approval, failed preconditions, and detected divergence cause rejection; after a durable or external boundary, divergent and indeterminate results remain visible and halt automatic continuation.

Trust basis. Meaningful persistent-outcome validation separates target specification, outcome evidence, and decision recording. The task owner or upstream application supplies A , a registered backend observation interface establishes E , and the runtime compares them and persists the decision. Allowing the agent to derive A from its own call or report E would reduce validation to self-attestation. The approving party, runtime decision core, and registered observation interface therefore carry these respective responsibilities; the agent, proposed operation, and action-triggered backend logic are the objects under validation.

III. PERSISTENT-OUTCOME VALIDATION PROTOCOL

EffectMatch implements the outcome classification in Section II through an *Effect Commit Contract* that connects application approval, execution evidence, and continuation control for one execution occurrence. The protocol checks approval before execution, compares the resulting effects at the backend’s control boundary, and saves the decision for subsequent workers. Figure 2 illustrates how these mechanisms cooperate on an invoice update.

A. Protocol Overview

Figure 2 follows the invoice update from application approval to rollback and a saved rejection. The three parts below correspond to the Introduction’s problems; circular badges locate the protocol steps in the figure.

1) Keep approval valid for the current execution. At **1**, the application approves changing invoice 4711 from *pending* to *paid*, with no notification-table insertion. The agent proposes the SQL command. Before executing it, the runtime binds approval to the current task state and one execution occurrence,

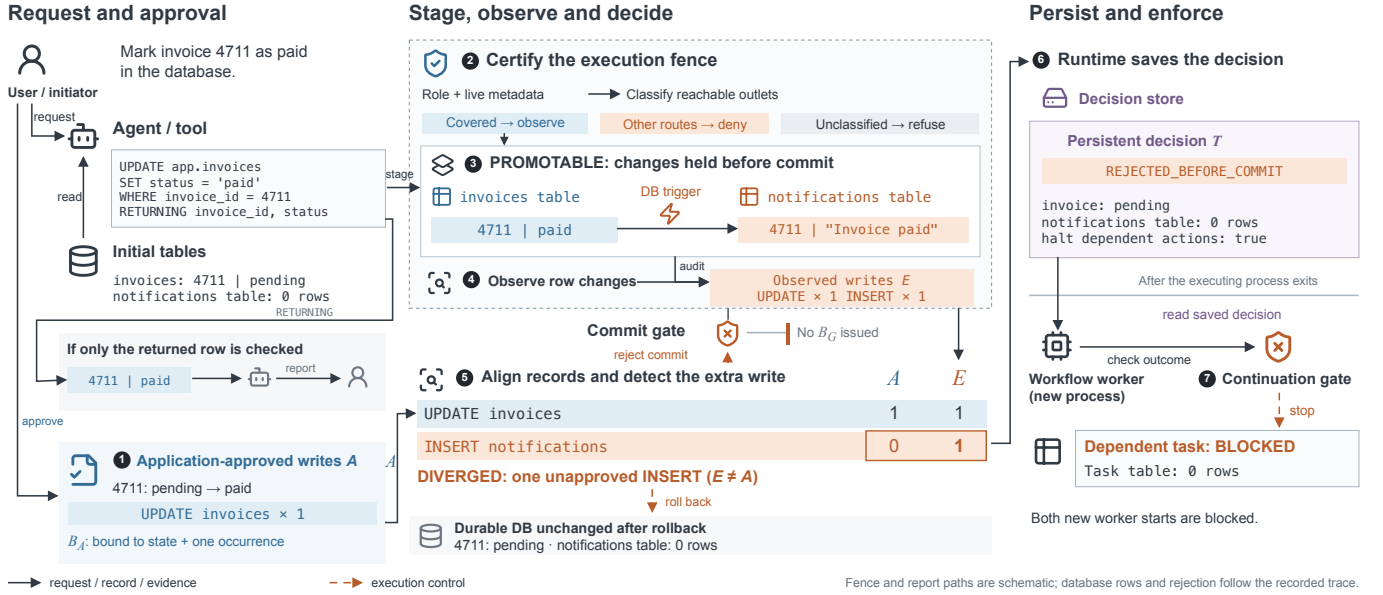


Fig. 2. EffectMatch illustrated through invoice 4711 on a backend that holds changes for inspection before commit (Table I). An unapproved trigger write causes rejection before commit; the saved decision blocks dependent work in later processes. A matching candidate receives a one-use commit permit B_G after validation; none is issued in this rejected trace. Circular labels 1–7 locate the protocol steps.

providing the authority against which this attempt’s effects will be checked. Section III-B describes the binding and its validation.

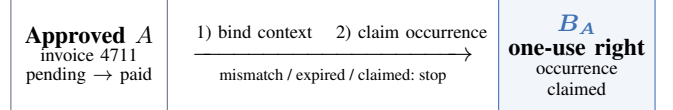
2) Establish the actual effects before accepting the result. At ②, the runtime checks the executing role and live backend configuration to certify that reachable routes to persistent effects are covered or blocked. An unclassified route prevents protected execution. Here, the scope covers both tables and the trigger path between them. At ③, the command runs inside a private transaction held open for inspection. Both the invoice update and trigger insertion remain reversible. At ④, transaction-local audit records expose the additional insertion alongside the requested update.

At ⑤, the runtime compares observed effects E with approved effects A by operation, relation, and write count. Both contain one invoice update; only E contains the notification insertion. The held candidate is DIVERGED, so the runtime rolls back both writes: the durable invoice remains *pending*, and the notifications table remains empty. The command’s successful return within the transaction therefore does not imply an accepted commit. Section III-C develops the observation and enforcement mechanisms for each backend capability.

3) Preserve the decision across process boundaries. Later workers must learn of the rejection before acting on the command’s apparent success. At ⑥, the runtime records a pre-commit rejection and a halt decision. Here, DIVERGED classifies the candidate’s mismatch; the saved rejection records its disposition. After the executing process exits, two fresh worker starts read this decision. The continuation gate at ⑦ blocks dependent execution, leaving the task table empty. Section III-D explains the journal’s role in recovery and continuation.

B. Binding Approval to One Execution

To close the approval-to-execution gap, EffectMatch turns the approved content into a state-bound, one-use execution right through two steps: binding its context and claiming its occurrence.



The application expresses the task requirements in an *Effect Manifest*, specifying target objects, permitted changes, and state that must remain unchanged. Pre-execution state and business rules determine the concrete approved result. Our independent-approval experiments use hand-authored task specifications and separate business calculations to construct this content.

For example, a request to cancel unshipped items specifies an order and the cancellation scope. Business rules select eligible items from their initial status, determine their cancellation status, refund amount, and refund method, and update the order status accordingly. Shipped items and other business data remain unchanged. The same cancellation rule takes different orders and initial states as inputs to construct each task’s approval. Once the user or application policy confirms the manifest, it becomes the reference for checking the agent’s execution.

Let M denote the confirmed manifest and p the registered effect profile. The profile specifies a canonical representation for comparing approved and observed changes. We denote its normalization by Canon_p and construct

$$A := \text{Canon}_p(M). \quad (2)$$

Profile p fixes the comparison rules in advance. Normalization standardizes the representation of operations, targets, results,

and multiplicities while preserving the targets and result constraints specified by the approval. The runtime renders the normalized content for approval and binds that same content in a signed license L , so the approving party and the execution check refer to the same outcome. Deployment frame F fixes the matching profile, protected boundary, and trusted runtime configuration; validation of L under F establishes the authority of A for the binding steps below.

The first step binds A to the approving source, current task state, plan revision, adapter, policy, expiry, task instance, and transaction or request. We denote this current, one-use authority binding by B_A . A changed target, value, revision, state, adapter, or execution context invalidates the old approval. For example, if payment of invoice 4711 is approved and its vendor is then blocked, the changed task state prevents the old approval from authorizing that payment.

The second step atomically claims the occurrence assigned to this execution before the first effectful stage or irreversible dispatch. A stable approval identifier maps to one runtime bundle rather than minting another right on each presentation. Stale state and expiry fail before the claim; a competing worker, retry, or replay finds the occurrence already spent. A crash after the claim leaves it spent, so failure cannot recreate authority for a duplicate effect.

C. Validating Persistent Outcomes at Backend Boundaries

Closing the second gap requires the runtime to establish the actual persistent outcome and turn its comparison with approval into a control decision at the backend boundary.

a) Certifying the execution boundary: A profile declaration names the intended observation scope; certification checks whether the executor's reachable persistence and dispatch routes have corresponding controls. As Figure 3 illustrates, the adapter combines the live execution identity and backend metadata with the host and network restrictions in force to classify these routes. Direct and trigger-induced writes to registered tables are covered by the observer; prohibited network access is blocked by enforcement; a route with no supported observation or blocking mechanism remains unclassified. The runtime issues a boundary certificate only when every reachable route has a supported disposition. An unclassified outlet prevents issuance and stops the operation before protected execution begins.

The PostgreSQL path certifies a preregistered, restricted deployment configuration. The adapter accepts its supported DML or fixed operation entry points and checks the registered relations, triggers, functions, and their permissions and configuration. The relation scope includes tables that supported trigger paths may modify, with protected write auditing on those relations. This scope is established by the deployment and includes relations that the current approval does not request to change.

A trusted metadata connection checks the live configuration. Unregistered functions, unsupported extensions, and reachable foreign-data or asynchronous-notification paths prevent admission; deployment isolation supplies the network restrictions outside the database. The certificate binds the outlet classi-

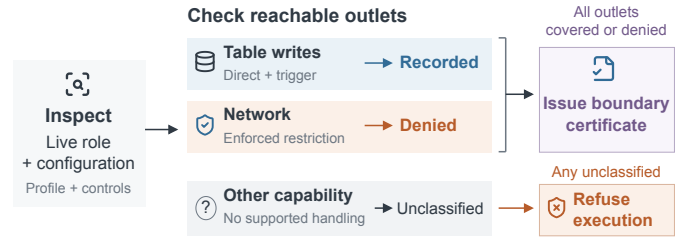


Fig. 3. Execution-fence certification: covered or denied outlets permit certification; any unclassified outlet prevents issuance.

fications and configuration, credential-provisioning, and enforcement digests to the backend, adapter, execution identity, and occurrence. The execution connection retains its restricted privileges. In the invoice example, the notification insertion therefore enters E even though it is absent from A .

b) Establishing the actual outcome: The records used to construct E come from execution at the protected backend boundary. In the PostgreSQL path, EffectMatch runs the action inside a rollback-capable private transaction and keeps that transaction open as the candidate. The registered relations are instrumented with write-recording triggers. A direct write and a write induced by an application trigger both produce audit records carrying the stage identifier and the affected relation and operation. Before collecting the records, the adapter executes `SET CONSTRAINTS ALL IMMEDIATE`, forcing deferred constraint triggers to run inside the held transaction. The observer can then collect their induced writes together with those already produced by the command.

For public database tasks using a net-state comparison, the adapter reads the registered data before execution and in the held candidate, then cross-checks their differences against this execution's write audit. These changes form E , which is compared with the approved outcome.

Let R_a denote the change records collected and attributed to execution a . Using the same profile as the approved manifest, the observer constructs

$$E := \text{Canon}_p(R_a). \quad (3)$$

The manifest and backend records may have different input formats; Canon_p denotes their respective mappings into the profile's common effect representation. The mapping preserves repeated effects and the comparison fields required by the profile. In Figure 2, the command returns invoice 4711 as *paid*, while the audit records contain both the invoice update and the notification insertion. The relation-write illustration shows their operations, relations, and multiplicities, which expose the extra insertion. Value-sensitive profiles additionally retain the result values or value digests in their comparison keys. For the held transaction, E describes candidate changes awaiting commit, so detecting a mismatch still leaves both writes reversible.

Constructing E organizes the available records; establishing $\text{Complete}_p(E)$ determines whether they cover the execution within the registered boundary. The certified profile fixes ∂_p before execution by registering the state objects, result-propagation paths, and evidence channels it covers and by

ruling out unclassified outlets for that executor. The observer must also establish the required evidence coverage and attribute the records to this execution. In Figure 2, the invoice and notification relations, the direct and trigger paths, and their held-transaction changes belong to the same observation scope. Their records therefore enter one E . Boundary certification and execution evidence jointly support completeness; a missing required channel or unresolved attribution leaves the result indeterminate.

Once completeness is established, an extra element in $E - A$, a missing element in $A - E$, or a changed result represented by the profile yields divergence. The PostgreSQL runtime locks the registered relations before reading the initial state and holds these locks until commit or rollback, stabilizing the data used in the comparison. Administrative role grants, function implementations, and external isolation are deployment-managed configuration; this path requires them to remain stable throughout protected execution. The final gate reads fresh metadata and compares the resulting certificate with the one bound to the permit, rejecting changes visible at that check. It also rechecks transaction identity, candidate state, and audit records before committing the same transaction. A failed check rolls back the held transaction. Non-transactional sequence residue is bounded separately and is not included in a zero-relational-residue claim.

c) *Enforcing the comparison:* Once E has been established, backend capability determines whether a mismatch can be rejected before durability or must be classified and contained afterward. Every mutating adapter declares one capability class, and admission checks that declaration against the interface the adapter actually controls. All classes claim the authorized occurrences before the first effectful stage or dispatch. Immediately before promotion or dispatch, the adapter revalidates state, authority, and its own identity, then consumes a permit bound to that particular stage or request. The capability class determines what can be checked before this gate and what can only be learned afterward.

Some backends let EffectMatch hold and observe the same candidate that it later promotes; we call this capability **PROMOTABLE**. The runtime controls the write credential or commit entry point, revalidates state and authority, and requires complete observation before promotion. A mismatch is discarded before durability; an equal candidate receives a stage-bound B_G and may be promoted. Database transactions, atomic file replacement, and server-side draft/promote interfaces can provide this shape.

Some remote backends cannot stage a candidate for later promotion but support a request path exclusively controlled by EffectMatch; we call this capability **MEDIATED**. Adapter admission requires an exclusive credential path, deterministic request generation from the approved manifest, and an idempotency mechanism that prevents repeated requests from duplicating effects. The adapter constructs the request from A , binds it in B_G , and consumes the permit before dispatch. The agent or an ordinary wrapper can therefore neither bypass the gateway nor add unapproved request fields. Post-dispatch evidence supports exact, diverged, or indeterminate classification under the same completeness requirements.

TABLE I
CAPABILITY-SPECIFIC CONTROL POINTS. EVERY PATH CLAIMS ITS ASSIGNED AUTHORIZATION OCCURRENCES BEFORE STAGING OR DISPATCH; A LATER PERMIT BINDS THE CHECKED STAGE OR REQUEST.

Class	Control point	Supported outcome
PROMOTABLE	inspect a held candidate before durability	mismatch or incomplete evidence: reject; confirmed equal promotion: exact; commit issued but result unestablished: indeterminate
MEDIATED	exclusive gateway; manifest-derived requests and idempotency; read back afterward	complete and equal: exact; complete and unequal: diverged; incomplete evidence or unresolved response loss: indeterminate
OPAQUE	constrain requests through the adapter; inspect available evidence afterward	complete and equal: exact; complete mismatch: diverged; incomplete evidence: indeterminate; no rollback of realized effects

For external calls that neither support staging nor meet the full gateway-control conditions above, we use the **OPAQUE** path. The adapter constrains requests sent through it and binds them to approval, then classifies the outcome using available post-execution evidence. Complete evidence matching approval permits continuation; a complete mismatch records divergence; insufficient evidence records uncertainty. The latter two outcomes block dependent execution, and this path provides no rollback of effects already realized.

Table I connects effect classification to execution outcomes. An exact candidate is recorded as successfully committed only after promotion is confirmed. Remote readback establishes the current state of registered objects; recording an exact execution outcome additionally requires matching approval and satisfying the coverage and attribution conditions of Definition 2. A failed check before commit or dispatch instead produces rejection, as in Figure 2. After the operation may have taken effect, the journal records divergence for a complete mismatch or uncertainty for an unresolved outcome. The journal thus records how execution ended, while the effect classification describes whether the observed candidate or outcome matched approval.

Every backend path must preserve its decision for later workers, so that rejection, divergence, or uncertainty continues to block dependent execution after the detecting process exits.

D. Persistent Recovery and Continuation

The capability-specific paths pass their execution decisions to later processes through the terminal journal. If the process exits before a database candidate commits, the uncommitted transaction rolls back when its connection terminates. An exit after business-state persistence but before terminal persistence leaves an execution requiring reconciliation. A new worker consults the existing terminal to decide whether work may proceed; a missing terminal keeps continuation blocked pending reconciliation. An occurrence already claimed remains spent.

The journal is also the release point for continuation. Schedulers, queue consumers, and recovery workers consult it

before dispatching a dependent action. Delayed work remains part of the same occurrence only when a registered continuation presents the matching occurrence identifier, continuation identifier, and authority lineage, then records its terminal receipt before releasing dependent work. An independent reader, mismatched lineage, or replay does not inherit the original authority. Dependent actions run only after reading a persistent terminal confirming exact completion; a missing terminal or a rejected, diverged, or indeterminate outcome keeps continuation blocked pending resolution. A spent occurrence cannot authorize an automatic resend. A later compensation or retry is a new business effect and requires new application authority. At task scope, EffectMatch preflights known capability and preconditions before the first external write, preserves exact terminals for actions already completed, and reports which remaining effects require resolution.

Let T denote the journal record for this execution. Recording confirmed exact completion requires agreement among the approved outcome, observed outcome, authority claim, and final-gate permit:

$$T \text{ records confirmed exact completion} \implies \begin{aligned} & \text{EffectExact}_p(A, E) \\ & \wedge \text{AuthValid}(B_A, t_c) \\ & \wedge \text{GateValid}(B_G, t_g). \end{aligned} \quad (4)$$

AuthValid records the successful current-state claim of one occurrence at t_c , while GateValid records that the checked stage or request consumed its bound permit at t_g . Both describe completed checks, not reusable credentials. The permit must be consumed to complete the authorized transition; requiring it to remain unspent would make a successful terminal self-contradictory.

Algorithm 1 consolidates these three stages into one capability-aware execution path. For a profile requiring outlet certification, CHECKBOUNDARY obtains a certificate before staging and VALIDATEANDCONSUME recaptures it at the final gate, together with the live-stage or request checks; other profiles use their declared admission checks. Every return passes through FINISH, which durably writes T before releasing an exact execution's dependents or blocking continuation for any other result.

E. Prototype Realization

The prototype is a Python runtime with registered adapters, effect profiles, and observers. Its mediated HTTP adapter attaches idempotency keys and available state preconditions to manifest-derived requests. In the STATE-Bench live-agent integration, tool calls first modify a private copy of the task state; the runtime then checks the candidate and decides whether to save it. When the model finishes normally, the integration layer triggers any outstanding finalization check.

The implementation persists the approval-to-bundle issuance ledger, occurrence ledger, terminal journal, and divergence receipts. A stable source nonce lets the issuance ledger deduplicate each issuer's approval, while the occurrence ledger records its one-time use.

Schema-closed approval, bundle, and permit records are HMAC-authenticated under separate keys; content-addressed

Algorithm 1 Capability-aware execution of the Effect Commit Contract.

Require: action a , state s , frame F , signed license L , profile p , capability c
Ensure: on normal return, persistent terminal T and continuation decision

Authority gate

- 1: $(A, B_A, \text{authOK}, t_c) \leftarrow \text{VALIDATEANDCLAIM}(a, s, F, L, c)$
- 2: $(E, B_G, t_g) \leftarrow (\perp, \perp, \perp)$; $\text{outcome} \leftarrow \perp$
- 3: **if** $\neg \text{authOK}$ **then**
- 4: **return** FINISH(REJECT)
- 5: $(\text{boundaryOK}, \text{cert}) \leftarrow \text{CHECKBOUNDARY}(p, a)$
- 6: **if** $\neg \text{boundaryOK}$ **then**
- 7: **return** FINISH(REJECT)
- 8: **if** $c = \text{PROMOTABLE}$ **then**
- 9: *Promotable: inspect before durability*
- 10: $H \leftarrow \text{STAGE}(a, s)$; $(E, \text{complete}) \leftarrow \text{OBSERVE}(H)$
- 11: **if** $\neg \text{complete}$ **or** $E \neq A$ **then**
- 12: DISCARD(H); **return** FINISH(REJECT)
- 13: $B_G \leftarrow \text{BINDSTAGEPERMIT}(B_A, H, E, \text{cert})$
- 14: *Recheck boundary and candidate*
- 15: $(\text{gateOK}, t_g) \leftarrow \text{VALIDATEANDCONSUME}(B_G, p)$
- 16: **if** $\neg \text{gateOK}$ **then**
- 17: DISCARD(H); **return** FINISH(REJECT)
- 18: $r \leftarrow \text{PROMOTE}(H)$; $\text{outcome} \leftarrow \text{PROMOTIONOUTCOME}(r)$
- 19: **else**
- 20: *Remote: constrain before dispatch; classify after*
- 21: $q \leftarrow \text{REGENERATEORCONSTRAIN}(A, c)$
- 22: $B_G \leftarrow \text{BINDREQUESTPERMIT}(B_A, q, A, \text{cert})$
- 23: $(\text{gateOK}, t_g) \leftarrow \text{VALIDATEANDCONSUME}(B_G, p)$
- 24: **if** $\neg \text{gateOK}$ **then**
- 25: **return** FINISH(REJECT)
- 26: $(r, \text{mayHaveSent}) \leftarrow \text{DISPATCH}(q)$
- 27: **if** $\neg \text{mayHaveSent}$ **then**
- 28: **return** FINISH(REJECT)
- 29: $(E, \text{complete}) \leftarrow \text{READBACK}(c, r)$
- 30: $\text{outcome} \leftarrow \text{CLASSIFYREADBACK}(E, \text{complete}, A)$
- 31: *Persistent terminal before continuation*
- 32: **return** FINISH(outcome)

contract and observer digests identify the code and profile used for each decision. The trusted boundary contains the runtime, authority-key holder, approval channel, policy store, and registered adapters and observers.

IV. EVALUATION

We evaluate EffectMatch primarily on public database tasks and shopping, customer-support, and travel tasks. Controlled comparisons and ablations test outcome checking and execution binding, while live-agent experiments assess task completion across agent configurations. Further experiments examine continuation after process failures, cross-backend behavior, and runtime overhead. The evaluation addresses five questions:

- RQ1** Compared with call-level rules and conventional workflows, can EffectMatch detect outcome mismatches and prevent incorrect commits?
- RQ2** Which EffectMatch components are necessary?
- RQ3** Can EffectMatch preserve successful task completion and integrate with different live-agent configurations?
- RQ4** Do persistent decisions correctly govern continuation after failures and process restarts?
- RQ5** How does EffectMatch behave across backends and deployment conditions, and what is its runtime overhead?

A. Experimental Setup

Comparison conditions. Public-task comparisons use direct execution, conventional workflow controls, and AgentSpec

TABLE II

MAIN EXPERIMENTAL POPULATIONS AND REPETITIONS. EXECUTION COUNTS INCLUDE ALL CONDITIONS AND VARIANTS WITHIN EACH BLOCK.

Experiment block	Distinct tasks / cases	Repetitions per cell	Executions
PostgreSQL: independent approval	3 public tasks	3	198
AgentSpec: visible / hidden writes	same 3 public tasks	3	81
STATE: independent approval	12 public tasks	3	828
STATE: clean-reference comparison	206 public tasks	3	7,416
Live-agent integration	6 public tasks	3	108
Cross-process recovery	18 public tasks	5	540
Runtime cost	same 18 public tasks	30 pairs per task	1,080
ToolSandbox	4 public scenarios	3	72
MySQL	8 constructed cases	5	240
GitHub	8 operation cases	5	120

A cell fixes the task or case, execution condition, and variant. Task sets overlap across rows. Recovery additionally starts 1,080 consumer processes. Certification, concurrency, and targeted reruns are specified with their respective results.

call-level rules [15]. Direct PostgreSQL execution also uses transactions; the AgentSpec condition uses its original rule interpreter with our SQL rule to check visible write operations and target relations. The STATE-Bench workflow condition executes tool calls in a private copy and discards that copy on a tool error. Ablations remove specified EffectMatch checks or bindings under the same task and fault conditions. Mechanism diagnostics additionally use *Complete-Effect Audit*, an instrumented complete-observation ceiling, which is reported separately from deployable controls.

Tasks and approval sources. The main experiments use database tasks from MCPMark¹ and shopping, customer-support, and travel tasks from STATE-Bench². For three selected database tasks and twelve business tasks, we independently construct approved outcomes before execution from public requests, initial state, and business rules. A broader comparison on 206 business tasks uses previously saved clean-execution results as targets to test checking with a supplied reference. ToolSandbox³ contributes four public scenarios; MySQL and GitHub use predeclared operation cases. Table II reports distinct tasks or cases, repetitions, and execution counts.

Execution and environment. Controlled experiments execute fixed SQL or replay public tool calls. Live-agent experiments generate independent call sequences with three configurations: a minimal DeepSeek API loop and DSH 0.1.5-rc.2, both using deepseek-flash with thinking disabled, and Codex CLI 0.154.0 using GPT-5.6 Sol with high reasoning. Each task supplies the same public request bundle and business-tool schemas to all three configurations, under direct and EffectMatch-integrated execution. Six tasks, three configurations, two conditions, and three repetitions yield 108 formal episodes. Business tools run in local simulators, while model calls use the actual APIs or CLI. Local experiments used an Apple M5 host and Python 3.12.13.

Outcome measurement. We measure normal task completion and incorrect persistent changes from the saved business state, and check continuation decisions through fresh processes that

TABLE III

PUBLIC-TASK COMPARISONS. CLEAN ROWS COUNT CORRECT COMMITS (↑); FAULT ROWS COUNT INCORRECT COMMITS (↓). EACH TASK/VARIANT IS REPEATED THREE TIMES.

(a) PostgreSQL: 3 independently approved tasks

Outcome	Direct	AgentSpec	EffectMatch
Clean ↑	9/9	9/9	9/9
Hidden extra write ↓	9/9	9/9	0/9
Missing row ↓	9/9	9/9	0/9
Altered value ↓	9/9	9/9	0/9

(b) STATE: 12 independently approved tasks

Outcome	Direct	Error-reject	EffectMatch
Clean ↑	36/36	36/36	36/36
Omit write ↓	36/36	36/36	0/36
Duplicate write ↓	9/36	9/36	0/36
Alter value ↓	36/36	36/36	0/36
Extra field change ↓	36/36	36/36	0/36

(c) STATE: 206 tasks with clean references

Outcome	Direct	Error-reject	EffectMatch
Clean ↑	618/618	597/618	618/618
Omit write ↓	354/618	138/618	0/618
Duplicate write ↓	138/618	138/618	0/618

Error-reject discards the private candidate on a tool error. Faults that leave the approved state unchanged remain in the denominators.

read persisted terminal records. Native benchmark scores provide post-execution assessment. Each condition decides using its assigned runtime evidence; raw calls, states, and decisions are retained in the experimental artifacts. The predeclared live-agent shopping comparison matches newly created cart items by their business fields and references, allowing their system-assigned identifiers to differ while preserving existing item identifiers. Cart-item list order is ignored; item multiplicities and the remaining business state are still checked.

B. RQ1: Can Outcome Checking Prevent Incorrect Commits?

An allowed call can still produce a result that violates approval. RQ1 compares call-level rules, conventional workflow controls, and EffectMatch on public tasks, asking whether they preserve correct execution while preventing incorrect commits.

¹<https://github.com/eval-sys/mcpmark>

²<https://github.com/microsoft/STATE-Bench>

³<https://github.com/apple/ToolSandbox>

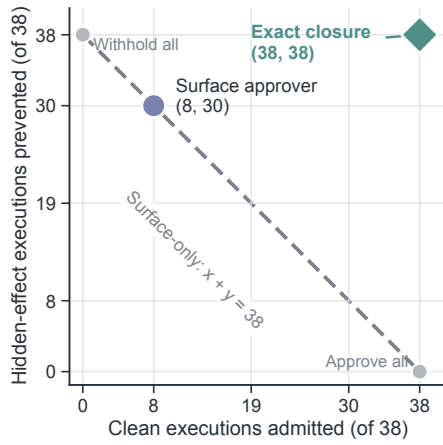


Fig. 4. Safety-utility boundary over 38 frozen STATE-Bench pairs. GPT-5.6 Sol (ultra) reaches (8, 30); exact post-execution closure reaches (38, 38).

Table III first reports independently approved database and business tasks, then extends the comparison to 206 tasks with saved clean-execution references. Figure 5 uses separate mechanism cases to explain the differences between evidence levels.

The three database tasks cover customer migration, employee hierarchy management, and sales/music charts. AgentSpec’s SQL rule checks visible write operations and target relations: in a separate explicit-write control, it rejects all nine unapproved writes. However, when the submitted SQL stays unchanged and fixture triggers introduce an extra write, a missing row, or an altered value, the rule has no resulting-row evidence on which to reject. EffectMatch compares candidate changes with the independently specified approval before commit and rejects all 27 mismatching executions, preserving the initial relation data. All conditions complete the clean cases. The distinction is whether the transaction should commit, rather than whether execution uses a transaction.

The twelve business tasks separate tool status from business correctness. Omission and duplication modify native calls; altered values and extra field changes are injected into the candidate snapshot after replay. These changes can leave an incorrect state without a tool error, so error-triggered rollback still accepts them. Duplication also exposes unnecessary rejection: nine executions change the required outcome, while 27 still reach the approved state. EffectMatch rejects the nine mismatches and commits the 27 matching candidates. Error-reject accepts all nine mismatches yet blocks twelve matching candidates across four tasks. Across the four fault types, EffectMatch rejects all 117 mismatches, representing 39 task/variant cases repeated three times. Checking the realized state separates harmful changes from calls whose repetition leaves the result intact.

The broader clean-reference comparison preserves this distinction. Error-reject reduces incorrect omission commits but misses every state-changing duplication in Table III(c). It also withholds 21 correct clean outcomes because their trajectories recover from an intermediate tool error. EffectMatch retains all 618 clean outcomes and rejects the mismatching candidates. Ineffective omissions and duplications remain eligible for

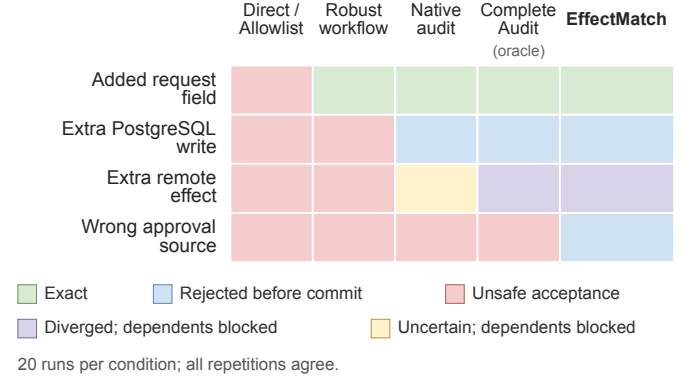


Fig. 5. Four mechanism scenarios from the full matrix: 20 runs per condition with identical within-cell outcomes. Direct and Allowlist ran separately and share a column. Complete Audit is the instrumented observation oracle. Diverged and uncertain outcomes block dependents without undoing the remote effect.

commit because their final state still matches the reference.

The mechanism diagnostics distinguish what each source of evidence adds. Here, Direct trusts the API response; Allowlist adds parameter, schema, and resource checks. Robust Workflow adds request constraints, state preconditions, readback, and transaction/retry controls; NativeAudit additionally uses ordinary backend audit evidence. Complete-Effect Audit receives the complete protected-effect set and a locally derived expected set. All six conditions complete the clean control in 20/20 runs each; Figure 5 displays four differentiating scenarios. The full eight-scenario matrix remains in the experimental artifacts.

Robust Workflow prevents the extra request field by constraining dispatch. NativeAudit additionally exposes the hidden PostgreSQL write and enables rejection before commit. For an already-realized remote effect, NativeAudit returns uncertainty, while complete observation establishes divergence; both stop dependent work. Finally, substituting the approval source leaves the observed effects unchanged, so even Complete-Effect Audit accepts the local match. EffectMatch rejects the untrusted source. Given the same approved target and complete observation, an in-transaction postcondition check can also reject the extra write. These comparisons show how outcome evidence changes commit decisions. RQ2 then examines whether approval still belongs to the current execution, whether the committed object is the checked candidate, and whether the saved decision continues to govern later work.

Figure 4 illustrates the corresponding information limit on 38 clean/hidden-effect pairs with identical pre-execution packets. The surface approver evaluates these packets in one tool-free batch; its six denials and 24 escalations both withhold execution. The same surface decision applies to both members of each pair: admitting x clean cases prevents at most $38 - x$ hidden counterparts. Post-execution observation distinguishes the pairs and reaches (38, 38).

C. RQ2: Which Components Are Necessary?

RQ1 showed how outcome checking changes commit decisions. RQ2 removes individual checks or related binding

TABLE IV
 PAIRED ABLATIONS GROUPED BY MECHANISM. PUBLIC TASK/VARIANT
 PAIRS HAVE THREE REPETITIONS; DEDICATED CASES HAVE 20.
 OUTCOMES REPORT COUNTS FOR FULL EXECUTION → REMOVAL.

Removed check	Case	Full → removal
Result comparison	PG tasks	27 reject → 27 wrong commits
	STATE tasks	117 reject → 117 wrong commits
Complete E	Extra write	20 reject → 20 extra-write commits
Approval source (B_A)	Source swap	20 reject → 20 wrong-source approvals accepted
Execution binding	Replay	20 reject → 20 duplicate effects
	PG occurrence	9 reject → 9 wrong-instance commits
	STATE occurrence	36 reject → 36 wrong-instance commits
Durable hazard (T)	Continuation	20 block → 20 dependent actions
Candidate binding (B_G)	Replacement	20 reject → 20 wrong-candidate commits
Final boundary check	Certificate reuse	20 reject → 20 stale-boundary commits
	PG joint recheck	9 reject → 9 stale-boundary commits

PG and STATE use three and twelve public tasks. Occurrence cases remove occurrence/pre-state binding. The two boundary rows remove fresh certificate capture and the combined boundary/audit recheck, respectively.

checks to identify where incorrect results or invalid approvals are admitted. Public-task ablations test result comparison, execution binding, and final rechecking; targeted cases isolate complete observation, approval provenance, persistent terminals, and candidate binding. Table IV pairs full EffectMatch with each removal.

Disabling result comparison commits all 27 incorrect database candidates and all 117 incorrect business-task candidates, while full EffectMatch rejects them. The other execution conditions remain unchanged, isolating whether the realized result is checked against approval. The observation ablation separately tests the comparison’s input: omitting the additional write from the observed effects makes an incorrect result appear to match approval. The provenance case tests the target instead. Its effects match a locally supplied target, but that target comes from an untrusted source. Full EffectMatch rejects it through the approval-source check in B_A .

Execution binding checks whether approval belongs to the current occurrence. Across three database and twelve business tasks, the candidate business result remains correct while the occurrence identifier changes. Full EffectMatch rejects nine and 36 executions, respectively; removing occurrence and pre-state binding admits all of them. Matching results and valid approval ownership are therefore independent conditions. The replay case shows the consequence of reuse: removing one-use enforcement produces an additional external effect in every repetition. The persistent terminal T carries the decision into subsequent execution. Without its durable hazard record, previously detected divergence is lost and a dependent action

is dispatched. Full EffectMatch rejects replay and preserves the divergence that blocks continuation.

Candidate binding ensures that the committed object is the one already checked. In the file-backed invoice case, the runtime checks a *paid* candidate and issues a valid permit; a second candidate for the same action then carries *cancelled*. Removing the B_G candidate-binding checks at both runtime and adapter lets that permit commit the replacement, whereas full EffectMatch rejects all substitutions. Signature, expiry, current policy, base-state validation, and one-use enforcement remain active. Candidate binding ties the validation decision to the specific result selected for commit.

Final rechecking also establishes whether the execution environment still satisfies certification. The database case grants temporary-table permission after staging, changing the permission boundary while leaving the protected relation catalog unchanged. Reusing the initial boundary certificate admits commit; fresh certification detects the change and rejects. The retained catalog check does not cover this database-level privilege. Committed rows still match approval: the failure is admission after certification conditions cease to hold. Across the three public database tasks, disabling the final boundary/audit recheck likewise admits all nine post-authorization deployment changes. That ablation removes the combined final recheck, while the targeted case isolates fresh certificate capture. The candidate-binding and certificate-capture studies both commit every clean control (20 per condition), and follow-up permit replays produce no second commit.

D. RQ3: Live-Agent Task Completion and Integration

The preceding questions examined how incorrect results are blocked and why the individual checks are necessary. RQ3 asks whether outcome checking can integrate with live agents while preserving successful task completion. We select six public STATE tasks covering shopping-cart changes, order cancellations, and travel-booking updates, and run DeepSeek API, DeepSeek+DSH, and Codex configurations. Each task has three repetitions under both direct execution and EffectMatch integration, yielding 108 independently generated live episodes. Business tools operate local simulators, and approval targets are independently established before model execution.

All 108 episodes produce candidate business states that match their approval targets. Table V reports whether the correct result is durably saved. Direct execution completes throughout; integrated DSH and Codex also complete throughout. For DeepSeek API, correcting finalization handling and rerunning the affected episodes produces correctly saved results for the remaining task repetitions.

These tasks combine required changes with state that must be preserved. The shopping task permits gift wrapping only for the specified product; the order task must retain already-shipped items; and the travel task must preserve booking preferences when changing a flight. Correct completion therefore requires both the requested changes and preservation of unaffected state. The observed candidates satisfy these requirements, and EffectMatch’s checks retain the corresponding paths to successful execution.

TABLE V

CORRECT DURABLE OUTCOMES ON SIX LIVE-AGENT TASKS, WITH THREE REPETITIONS PER CONFIGURATION AND CONDITION.

Agent configuration	Direct	With EffectMatch
DeepSeek API	18/18	18/18*
DeepSeek+DSH	18/18	18/18
Codex	18/18	18/18

*Original integrated runs completed 51/54. The marked cell combines 15 original successes with three successful targeted reruns after host-triggered finalization was added at normal agent completion.

TABLE VI

PERSISTENCE AND CONTINUATION AFTER PROCESS EXIT. EACH ROW COUNTS 90 EXECUTIONS: 18 TASKS WITH FIVE REPETITIONS.

Scenario	Correct state	Terminal	Release
Normal completion	90	90	90
Exit after staging	0	0	0
Exit after state save, before terminal	90	0	0
Exit after terminal save	90	90	90
Omit first write; exit after staging	0	0	0
Omit first write; exit after terminal save	35	90	35

Correct state counts saved results matching the reference. Terminal counts include success and rejection. Release uses a harness acknowledgement; two fresh readers produce at most one saved acknowledgement per execution.

The three configurations organize tool execution through a minimal API loop, the DSH harness, and Codex CLI, respectively. EffectMatch follows the same integration principle in each: business tools first form a candidate state, and the runtime compares it with independently approved targets before saving it. This places outcome checking at the business-state persistence point across different agent frameworks, without reading the model’s internal reasoning. The results support the feasibility of this integration on the tested tasks and configurations.

E. RQ4: Continuation after Failures and Process Restarts

RQ3 examined successful task completion with live agents. RQ4 asks whether a new worker can correctly decide what may continue after the executing process exits. We select six public multistep tasks from each of shopping, customer support, and travel, replay their tool calls, and use previously saved clean-execution results as comparison targets. Six normal or interrupted scenarios, each repeated five times per task, produce 540 execution processes. After each exits, two fresh processes read the saved business state and terminal records, yielding 1,080 continuation checks.

Table VI shows how the exit point affects persistence and continuation. Normal completion and exit after terminal persistence both permit continuation; exit while the candidate remains staged preserves the initial state and stops subsequent work. No case releases work without a valid result, and the two fresh readers produce no duplicate acknowledgement records.

TABLE VII

CROSS-BACKEND OUTCOMES. COUNTS ARE PER VARIANT; EACH OPERATION OR SCENARIO IS REPEATED AS INDICATED.

Backend / coverage	Clean	Perturbed
MySQL audit prototype 8 operations $\times$ 5	40/40 correct commits	Extra write: 40/40 rejected Omission: 40/40 rejected
ToolSandbox held runtime 4 scenarios $\times$ 3	12/12 correct commits	Extra reminder: 12/12 rejected Omission: 12/12 rejected
GitHub readback 8 operations $\times$ 5	40/40 fields match	Changed title: 40/40 mismatches detected; blocked No readback: 40/40 blocked

MySQL and GitHub use predeclared operation cases; ToolSandbox uses public tasks with request-derived approval. MySQL implements comparison in a database-side audit procedure. GitHub reports registered-field readback; blocking means withholding the local continuation acknowledgement.

Recovery checks both the successful terminal and its bound durable state. A saved success therefore survives the producing process, and a new worker can use it to continue. The interval between business-state persistence and terminal persistence exposes a gap: all 90 executions leave the correct state, but continuation is withheld because the terminal is missing. The new workers in this experiment read existing terminals without reconstructing missing records, so these executions await reconciliation before continuation. Release is represented by a unique acknowledgement record in the harness.

The omitted-write results show that continuation depends on the realized state. In 35 of 90 executions, omitting the first write leaves the final result unchanged, so checking permits persistence and continuation. The remaining 55 candidates mismatch the target; the runtime saves rejection, and subsequent workers stop. This carries forward the outcome rule from RQ1 and RQ2: a changed execution trace need not imply an incorrect business result.

The independently approved PostgreSQL customer-migration task exhibits the same recovery behavior. Five repetitions at each exit point yield 0/5 continuation acknowledgements before commit, 0/5 after commit but before terminal persistence, and 5/5 after terminal persistence. In the last group, two fresh readers still produce one acknowledgement per execution. This extends the recovery observation to a real database transaction path.

The original 80 cases test how these decisions apply to linear, fan-out, fan-in, and mixed dependency structures, crossing four disruptions with five instances per topology. A preflight blocker stops the plan before any effect; a mid-run failure preserves completed predecessors while stopping the failed node and dependent successors. In the mid-run failures, fan-in cases retain two completed predecessor effects per instance, while the other topologies retain one. Changed plans and stale or replayed approvals are rejected, and all 80 handoffs match durable state. Failure position and dependencies determine which work survives; the saved record carries that decision to subsequent execution.

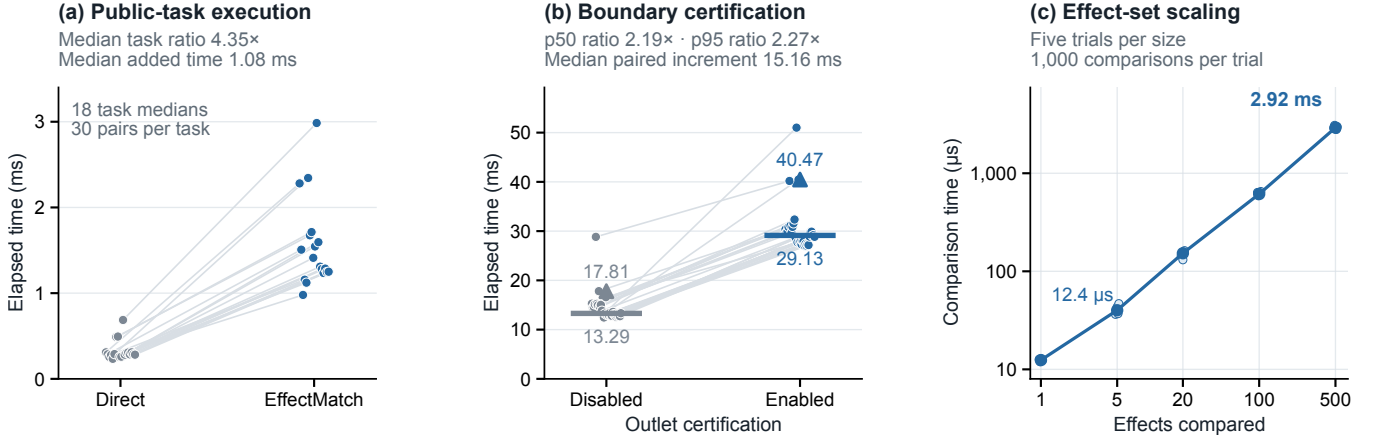


Fig. 6. Runtime cost at three levels. (a) Each connected pair shows one STATE task’s median over 30 alternating direct/held pairs; annotations summarize the 18 task-level ratios and differences. (b) Thirty pairs of five-effect PostgreSQL executions isolate added outlet certification, retaining effect comparison and one-use permits in both conditions. Dots are runs, bars are medians, and triangles are nearest-rank p95. (c) In-memory comparison over 1–500 effects: small markers show five trials of 1,000 comparisons each, and the line joins trial medians.

F. RQ5: Cross-Backend Behavior, Deployment Conditions, and Cost

The preceding questions examine outcome checking, component contributions, live-agent integration, and continuation after failures. RQ5 asks how these checks transfer across backends, how backend control affects the response to a mismatch, and what the checks cost. Table VII extends pre-persistence checking to MySQL and ToolSandbox and adds a remote-state readback study on GitHub; Figure 6 separates public-task execution cost, PostgreSQL boundary-certification overhead, and effect-set comparison.

Checking before persistence. MySQL’s audit prototype and ToolSandbox’s held runtime compare candidate results before saving them; rejecting a mismatch preserves the initial state. Direct MySQL transactions commit every extra-write and omitted-statement variant: atomic execution alone does not decide whether the resulting changes were approved. In ToolSandbox, the extra reminder still passes all 12 native task-goal checks under direct execution. Comparing the approved and observed state changes also detects this unrequested action, even when the requested goal is satisfied.

Remote-state readback and continuation. GitHub provides an auxiliary test of remote-state readback and local continuation control. A fresh GET after the modifying request retrieves the registered issue fields: title, body, state, labels, and assignees. Matching fields produce a local continuation acknowledgement; an out-of-band title change or unavailable readback produces a saved decision that blocks this acknowledgement and automatic resend. These measurements concern the state at readback, without attributing the title change to the evaluated request. Previously applied remote changes remain in place.

Certifying execution conditions. The component study checks admission and revalidation across 21 constructions, each repeated five times, covering foreign-database, host/network, asynchronous-signal, configuration-drift, and continuation conditions. The integrated PostgreSQL 17.6 study then checks five scenarios with 20 trials each; independent

database readback agrees with runtime decisions in all 100 trials. Clean operations commit, effect comparison rejects additional writes, initial certification refuses an exposed host capability, fresh final-gate metadata catches a later capability grant, and a consumed permit prevents a second commit. Role grants expose the tested host capability for metadata inspection without invoking a host-file operation. These cases separate checking the business outcome from checking the execution conditions under which that outcome is observed.

Public-task execution cost. Figure 6(a) measures 30 alternating direct/held pairs for each of 18 STATE tasks, totaling 1,080 timed executions, all with correct results. Across task medians, the median held/direct ratio is 4.35 $\times$ and the median added time is 1.08 ms. Direct task medians are below one millisecond, so a small absolute increment still produces a substantial ratio. Timing includes native replay, runtime construction, comparison, permit handling, and durable SQLite state selection, with input preparation and two warmups per task and condition excluded. It measures the local execution path, excluding model calls and remote-service latency.

Boundary-certification cost. Figure 6(b) alternates outlet certification off and on for 30 pairs of the same five-effect PostgreSQL operation. Both paths retain relational catalog checks, effect comparison, and one-use permits; two warmup pairs are excluded. Timing covers preparation through commit, including metadata connections, with fixture setup and independent readback outside the timer. Certification raises median latency from 13.29 to 29.13 ms (2.19 $\times$), with a median per-pair increment of 15.16 ms. Initial and final certificate capture take 7.01 and 7.21 ms at the median, respectively, each opening a fresh metadata connection. Their contribution identifies connection and metadata access as the main optimization target while retaining a fresh final check.

Figure 6(c) shows approximately linear growth in comparison time over the tested range of 1–500 effects, reaching a median of 2.92 ms at 500 effects. This measurement isolates the computational cost of effect comparison; the execution paths also include staging, persistence, and metadata access.

TABLE VIII
EXECUTION CONTRACTS BY PRIMARY CONTROL OBJECT.

Contract	Controlled object	Decision and retained result
Action/request authority [2], [3]	proposed action or canonical request	authorize at tool admission or trusted-sink dispatch
Information flow [16], [17]	capability-controlled or labeled flow	permit or block the flow reaching tool use
Runtime rules [15]	trigger events, predicates, and interventions	check and intervene at specified execution events
Authority life-cycle [7], [8]	authority witness or authorization instance	validate at commit; CapLease retains durable issue/prepare/commit state
Agent transactions [18], [19], [20]	staged effects and execution footprint	validate, commit, roll back, compensate, or recover
EffectMatch	approved and observed outcomes of one execution	persist the comparison decision to govern commit, retry, and dependent work

G. Scope and Deployment Implications

Exactness within a certified profile. Execution-level exactness requires the profile’s declared coverage and execution attribution at its chosen comparison granularity (Section II-C). The GitHub study separately evaluates registered-state read-back and local continuation control. The tested PostgreSQL execution path additionally requires its outlet certificate to remain valid through the final gate. The certification matrix covers five representative PostgreSQL boundary classes, not an exhaustive enumeration of database, extension, or operating-system behavior. For opaque backends, an already durable effect can be classified and contained when the required evidence is available; undoing it requires separately authorized compensation.

Authority and extension. Deployment requires application-supplied approval and the trusted components in Section III-E. Adapters provide their profiles’ declared observation interfaces and execution controls, enabling outcome comparison and commit or continuation decisions. Gateway-mediated services also require control over credential and egress paths to route protected requests through the gateway. Coverage expands by specifying approved effects for new operations, registering observers, and validating adapter capabilities under the same contract. Cost results describe the tested execution paths.

V. RELATED WORK

Agent execution safety spans three related questions: what operation may be attempted, what effects that operation establishes, and whether later work may proceed. Prior work approaches this chain through threat benchmarks, runtime policy, authority lifecycle, and transactional recovery. Table VIII groups representative systems by the object their primary contract controls and follows that object to the relevant execution boundary and outcome.

Agent risks and state-based evaluation. AgentDojo evaluates prompt injection against agents operating tools over untrusted data, while ToolEmu uses an LM-emulated sandbox to expose long-tail execution risks [11], [21]. Agent Security Bench broadens the attack surface across prompts, tools, and memory, and AgentHarm tests whether an agent can carry a harmful request through a coherent multistep trajectory [22],

[23]. Beyond attack labels, τ -bench scores policy-guided tool use by comparing the final database state with an annotated goal state and measures reliability over repeated trials [24]. These benchmarks expose execution risks and establish the value of state-based assessment. EffectMatch places outcome comparison in the protected execution path so that its result directly governs commit and subsequent work.

Policy and runtime guardrails. CaMeL and Fides control how untrusted data may influence tool use through capability separation or information-flow constraints [16], [17]. AgentSpec expresses runtime constraints through structured rules comprising trigger events, predicates, and interventions; Progent enforces programmable least privilege over tool calls; ShieldAgent verifies action trajectories against explicit safety policies; and DRIFT combines dynamic rule checking with injection isolation [15], [25], [26], [27]. These methods provide different mechanisms for expressing and enforcing execution policy. EffectMatch studies a specific outcome constraint: compare actual changes with the result approved for this execution, within a boundary whose observation conditions have been established, and persist the decision governing commit and continuation.

Authority, consent, and proof. AIRGuard derives step-level authority and intercepts normalized actions, while SecureClaw authorizes a canonical request at a trusted sink [2], [3]. Temporary Authority revalidates freshness, dependency, and target evidence at commit; CapLease makes authorization issue, preparation, and consumption durable across retry and replay [7], [8]. Consent Integrity binds an approved presentation to the action that reaches a black-box agent boundary, and proof-oriented proposals carry planning or action certificates into later execution and audit [28], [9], [29]. UCON models ongoing conditions and mutable attributes during use, while macaroons attenuate delegated authority with contextual caveats [30], [31]. These approaches refine the source, object, and validity conditions of authority. EffectMatch binds the application-approved outcome to the current state and one execution, and checks that the approval still applies when comparing outcomes.

Transactions, effects, and recovery. Cordon stages task effects in shadow state and an effect outbox, then validates the composed execution flow before commit; Atomix records read/effect footprints and delays settlement until earlier conflicting work can no longer arrive; SagaLLM combines validation logs, checkpoints, and compensation for multi-agent workflows [18], [19], [20]. Verification-aware tool wrappers use postcondition checks, idempotency keys, and verify-before-retry to reduce duplicate remote actions under non-atomic failures [32]. Durable Functions gives stateful serverless workflows replay-based semantics, while AFT interposes on storage to provide atomic visibility across retried functions [33], [34]. These systems extend foundations in effects, transactions, optimistic validation, sagas, and idempotency [35], [4], [36], [37], [38], [39], [5]. EffectMatch takes the application’s prior approved outcome as the comparison target and connects outcome matching, current approval, and the continuation decision to the same execution.

Persistent-outcome validation. The protocol combines trans-

actional staging, outcome comparison, and persistent decision records to link application approval, the checked candidate, and continuation to the same execution. Backends supply either a staging-and-commit control point or request dispatch followed by readback; the protocol maintains the correspondence among approval, outcome, and continuation at these points. It builds on provenance [40], [41], [42], capability systems [43], [44], runtime enforcement [12], [13], and sender-constrained credentials [45].

VI. CONCLUSION

The persistent changes left by an agent operation must conform to the approval for that execution. Within a controlled execution boundary, EffectMatch binds application approval to the current state and one execution, checks actual changes, and persists the decision governing commit and subsequent work. The Effect Commit Contract connects approval, outcome comparison, and continuation so that the execution decision remains available after process exit or recovery.

On twelve public business tasks with approved outcomes independently specified before execution, EffectMatch preserved all clean executions and rejected the mismatches in all 39 task–fault combinations, each repeated three times. Extended evaluation covered reference-based comparison on 206 public tasks, live execution with three agent configurations, cross-process recovery, and different database and external-service backends. By making continuation depend on the persisted outcome decision, the protocol prevents executions that violate approval or remain unresolved from propagating as success.

ACKNOWLEDGMENT

OpenAI Codex and Anthropic Claude were used as author-directed tools for writing and revision support, code development, and schematic preparation. The authors reviewed all outputs and take full responsibility for the manuscript.

REFERENCES

- [1] T. South, S. Marro, T. Hardjono, R. Mahari, C. D. Whitney, A. Chan, and A. Pentland, “Position: AI agents need authenticated delegation,” in *Proceedings of the 42nd International Conference on Machine Learning*, vol. 267 of *Proceedings of Machine Learning Research*, pp. 82211–82231, 2025.
- [2] S. Qin, H. Zhuang, Y. Zhou, Y. Han, and X. Zhang, “AIRGuard: Guarding agent actions with runtime authority control,” *arXiv preprint arXiv:2605.28914*, 2026.
- [3] Y. Ma and S. Schmid, “SecureClaw: Clawing back control of LLM agents,” *arXiv preprint arXiv:2606.09549*, 2026.
- [4] J. Gray and A. Reuter, *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann, 1992.
- [5] P. Helland, “Idempotence is not a medical condition,” *Communications of the ACM*, vol. 55, no. 5, pp. 56–65, 2012.
- [6] H. Zhang, A. Cardoza, P. B. Chen, S. Angel, and V. Liu, “Fault-tolerant and transactional stateful serverless workflows,” in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pp. 1187–1204, USENIX Association, 2020.
- [7] I. Santos-Grueiro, “Temporary authority, permanent effects: Commit-time authorization for LLM agents,” *arXiv preprint arXiv:2607.10487*, 2026.
- [8] J. Xu, L. Fan, Z. Wang, X. Li, and H. Liu, “Beyond single-use tokens: Durable authorization state for replay-resistant LLM agent actions,” *arXiv preprint arXiv:2608.01710*, 2026.
- [9] S. Cohen, “Execution-witness binding: Proof-carrying authorization for LLM agent runtimes,” *SSRN preprint 6352164*, 2026.
- [10] Z. Wu, X. Liu, X. Zhang, L. Chen, F. Meng, L. Du, Y. Zhao, F. Zhang, Y. Ye, J. Wang, Z. Wang, J. Ni, Y. Yang, A. Xu, and M. Q. Shieh, “MCPMark: A benchmark for stress-testing realistic and comprehensive MCP use,” *arXiv preprint arXiv:2509.24002*, 2025.
- [11] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents,” in *Advances in Neural Information Processing Systems*, vol. 37, pp. 82895–82920, Curran Associates, Inc., 2024.
- [12] F. B. Schneider, “Enforceable security policies,” *ACM Transactions on Information and System Security (TISSEC)*, vol. 3, no. 1, pp. 30–50, 2000.
- [13] J. Ligatti, L. Bauer, and D. Walker, “Edit automata: Enforcement mechanisms for run-time security policies,” *International Journal of Information Security*, vol. 4, no. 1, pp. 2–16, 2005.
- [14] PostgreSQL Global Development Group, *PostgreSQL 17 Documentation: Overview of Trigger Behavior*, 2024. Accessed August 2026.
- [15] H. Wang, C. M. Poskitt, and J. Sun, “AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents,” in *Proceedings of the 48th IEEE/ACM International Conference on Software Engineering*, 2026. To appear.
- [16] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, “Defeating prompt injections by design,” *arXiv preprint arXiv:2503.18813*, 2025.
- [17] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, “Securing AI agents with information-flow control,” *arXiv preprint arXiv:2505.23643*, 2025.
- [18] Z. Chen, H. Liu, D. Xu, D. Dong, J. Li, B. Pu, and J. Zhai, “Cordon: Semantic transactions for tool-using LLM agents,” *arXiv preprint arXiv:2606.17573*, 2026.
- [19] B. Mohammadi, N. Potamitis, L. Klein, A. Arora, and L. Bindschaedler, “Atomix: Timely, transactional tool use for reliable agentic workflows,” *arXiv preprint arXiv:2602.14849*, 2026.
- [20] E. Y. Chang and L. Geng, “SagaLLM: Context management, validation, and transaction guarantees for multi-agent LLM planning,” *Proceedings of the VLDB Endowment*, vol. 18, no. 12, pp. 4874–4886, 2025.
- [21] Y. Ruan, H. Dong, A. Wang, S. Pitis, Y. Zhou, J. Ba, Y. Dubois, C. J. Maddison, and T. Hashimoto, “Identifying the risks of LM agents with an LM-emulated sandbox,” in *International Conference on Learning Representations*, vol. 2024, pp. 27031–27098, 2024.
- [22] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang, “Agent security bench (ASB): Formalizing and benchmarking attacks and defenses in LLM-based agents,” in *International Conference on Learning Representations*, vol. 2025, pp. 35331–35366, 2025.
- [23] M. Andriushchenko, A. Souly, M. Dziemian, D. Duenas, M. Lin, J. Wang, D. Hendrycks, A. Zou, Z. Kolter, M. Fredrikson, Y. Gal, and X. Davies, “AgentHarm: A benchmark for measuring harmfulness of LLM agents,” in *International Conference on Learning Representations*, vol. 2025, pp. 79185–79220, 2025.
- [24] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan, “ τ -bench: A benchmark for tool-agent-user interaction in real-world domains,” in *International Conference on Learning Representations*, vol. 2025, pp. 9965–10017, 2025.
- [25] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song, “Progent: Securing AI agents with privilege control,” *arXiv preprint arXiv:2504.11703*, 2025.
- [26] Z. Chen, M. Kang, and B. Li, “ShieldAgent: Shielding agents via verifiable safety policy reasoning,” in *Proceedings of the 42nd International Conference on Machine Learning*, vol. 267 of *Proceedings of Machine Learning Research*, pp. 8313–8344, PMLR, 2025.
- [27] H. Li, X. Liu, H.-C. Chiu, D. Li, N. Zhang, and C. Xiao, “DRIFT: Dynamic rule-based defense with injection isolation for securing LLM agents,” in *Advances in Neural Information Processing Systems*, vol. 38, pp. 83262–83290, Curran Associates, Inc., 2025.
- [28] X. Weng, “What you approve is what executes: Consent integrity for black-box LLM agents,” *arXiv preprint arXiv:2606.02668*, 2026.
- [29] Z. Wang, “Proof-carrying agent actions: Model-agnostic runtime governance for heterogeneous agent systems,” *arXiv preprint arXiv:2606.04104*, 2026.
- [30] J. Park and R. Sandhu, “The uconabc usage control model,” *ACM transactions on information and system security (TISSEC)*, vol. 7, no. 1, pp. 128–174, 2004.
- [31] A. Birgisson, J. G. Politz, U. Erlingsson, A. Taly, M. Vrabie, and M. Lentzner, “Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud,” in *Proceedings of the 2014 Network and Distributed System Security Symposium*, Internet Society, 2014.

- [32] I. K. Mansoor, A. Phadke, and P. Rana, “Verified tool calls improve LLM agent reliability under non-atomic failures,” *arXiv preprint arXiv:2608.02645*, 2026.
- [33] S. Burckhardt, C. Gillum, D. Justo, K. Kallas, C. McMahon, and C. S. Meiklejohn, “Durable functions: Semantics for stateful serverless,” *Proceedings of the ACM on Programming Languages*, vol. 5, pp. 1–27, Oct. 2021.
- [34] V. Sreekanti, C. Wu, S. Chhatrapati, J. E. Gonzalez, J. M. Hellerstein, and J. M. Faleiro, “A fault-tolerance shim for serverless computing,” in *Proceedings of the Fifteenth European Conference on Computer Systems*, EuroSys ’20, pp. 1–15, ACM, 2020.
- [35] J. M. Lucassen and D. K. Gifford, “Polymorphic effect systems,” in *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pp. 47–57, 1988.
- [36] C. Mohan, D. Haderle, B. Lindsay, H. Pirahesh, and P. Schwarz, “ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging,” *ACM Transactions on Database Systems*, vol. 17, no. 1, pp. 94–162, 1992.
- [37] H. T. Kung and J. T. Robinson, “On optimistic methods for concurrency control,” *ACM Transactions on Database Systems*, vol. 6, no. 2, pp. 213–226, 1981.
- [38] H. Garcia-Molina and K. Salem, “Sagas,” in *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, pp. 249–259, 1987.
- [39] P. Helland, “Life beyond distributed transactions: An apostate’s opinion,” in *Proceedings of the Third Biennial Conference on Innovative Data Systems Research (CIDR 2007)*, (Asilomar, CA, USA), pp. 132–141, 2007.
- [40] P. Buneman, S. Khanna, and W.-C. Tan, “Why and where: A characterization of data provenance,” in *International conference on database theory*, pp. 316–330, Springer, 2001.
- [41] T. J. Green, G. Karvounarakis, and V. Tannen, “Provenance semirings,” in *Proceedings of the twenty-sixth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*, pp. 31–40, 2007.
- [42] J. Cheney, L. Chiticariu, and W.-C. Tan, “Provenance in databases: Why, how, and where,” *Foundations and trends in databases*, vol. 1, no. 4, pp. 379–474, 2009.
- [43] J. B. Dennis and E. C. Van Horn, “Programming semantics for multi-programmed computations,” *Communications of the ACM*, vol. 9, no. 3, pp. 143–155, 1966.
- [44] J. H. Saltzer and M. D. Schroeder, “The protection of information in computer systems,” *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975.
- [45] D. Fett, B. Campbell, J. Bradley, T. Lodderstedt, M. Jones, and D. Waite, “OAuth 2.0 demonstrating proof of possession (DPoP),” RFC 9449, Internet Engineering Task Force, 2023.

Supplementary Material for Beyond Approved Actions: Runtime Validation of Persistent Outcomes in Agent Workflows

Haoran Zhang, Hengtong Zhang, Zhiyu Liang, Yu Yan, Decheng Zuo, and Hongzhi Wang, *Senior Member, IEEE*

This supplement provides construction examples, implementation details, full comparison results, and live-agent settings for EffectMatch. Sections S1 and S2 accompany the execution model and protocol in Sections II–III of the main paper. Sections S3 and S4 accompany its evaluation. All results come from the experiments reported in the paper; tables here expose additional cells or task-level detail. Section, table, and equation numbers beginning with S refer to this document.

S1. CONSTRUCTING APPROVED AND OBSERVED OUTCOMES

A. Inputs, Representation, and Independence

An approved outcome A fixes what the application accepts before the candidate executes. An observed outcome E represents the changes established from execution evidence. Both use the granularity of the registered profile p .

For a write-event profile, entries retain operations, targets, and multiplicities. For a net-state profile, entries describe changes between the initial and candidate states, retaining values or value digests. Thus two writes that restore the initial value can differ under event comparison while producing an empty net-state difference. The profile, rather than the trial outcome, selects this interpretation.

The independently approved experiments construct their targets from public task requests, initial business data, and hand-authored task specifications with independent business calculations. PostgreSQL contracts calculate requested row values without executing the candidate SQL. STATE contracts apply separate Python transformations without invoking native mutating handlers. The runtime receives the resulting approved content and its execution binding. Benchmark target constraints and post-execution scores are reserved for assessment after the durable decision. The 206-task STATE comparison has a different target source. It saves one separate clean native replay per task before running the compared conditions. That saved result supplies the reference for outcome checking. The comparison measures enforcement with a supplied target; the independently specified subset measures enforcement with a request-derived target. These populations retain their respective approval sources throughout the evaluation.

For the snapshot adapters, the approved target is a complete expected business snapshot. Its difference from the initial

TABLE S1
REQUEST-DERIVED CHANGES FOR ORDER ORD-7281. ALL OTHER
REGISTERED BUSINESS FIELDS REMAIN UNCHANGED.

Object / field	Initial	Approved
Order: status	partially delivered	partially cancelled
Case item: status	confirmed	cancelled
Case item: refund amount	null	35
Case item: refund method	null	split

snapshot gives the approved effect representation. The candidate snapshot is independently produced by native tools, and its difference from the same initial snapshot gives the observed representation. Unrequested business records retain their initial values in the target, so changing an unrelated field creates a mismatch. The comparison operates over the registered business namespaces; conversation transcripts are not treated as business-state effects.

B. Example: Partial Order Cancellation

The public customer-support task 150-spare_cancel_in_transit_fee_decline requests cancellation of the still-cancellable part of order ORD-7281. Its initial state contains a delivered phone and an unshipped phone case. The independent specification selects the unshipped case, preserves the delivered phone, and derives the cancellation and refund fields from the initial order. Table S1 shows the resulting changes. Amounts use the task’s currency units.

The order item is ITEM-10286. The calculation sets its refund to its initial unit price, 35, and its refund method to the order’s existing split-payment method. The order becomes partially cancelled because the delivered item remains unchanged. A native execution that produces these changes matches the target.

In the controlled altered-value variant, the harness increases the refund amount in the candidate snapshot by one after native replay. The resulting candidate still contains the requested cancellation, but its refund is 36. Value-sensitive comparison rejects the candidate and preserves the initial durable state. The injected value is an experimental fault; the runtime receives candidate state, not the fault label. This example also shows why the requested operation alone does not define A . A generic “cancel order” call does not specify which already-delivered item must remain, the permitted refund amount, or the required payment method. The independent application specification supplies these constraints before execution.

C. Example: Database Migration and Hidden Writes

For MCPMark’s `customer_data_migration`, the approval calculation reads the public import records and the initial database. It constructs the 200 requested customer rows, including their field values and defaults, before candidate SQL executes. Unrequested registered rows must remain unchanged. The evaluated command runs in a private PostgreSQL transaction, held open for inspection. The observer reads the registered relations before execution and in the held candidate, then reconciles those differences with protected, transaction-local write records. A hidden fixture trigger can additionally update the `Genre` relation while the submitted SQL stays unchanged. Because `Genre` belongs to the registered scope, its change enters *E* despite being absent from *A*. Missing-customer and altered-name fixtures instead remove or change an approved result. All three variants reach the same outcome-comparison step; rejection rolls back the held transaction.

The invoice illustration in Figure 2 of the main paper uses a smaller relation-write projection: one invoice update is approved, and a notification insertion is additional. The public database contracts retain row values as well. Relation-operation counts alone cannot distinguish two 200-row imports that differ only in a customer’s name.

D. Execution Binding and Stored Decisions

The approval binding B_A associates approved content with its source, task state, plan revision, profile, and execution occurrence. The final permit B_G associates a successful check with the particular held candidate or mediated request that may be committed or dispatched. A signed permit for one candidate cannot be transferred to another candidate merely because both belong to the same task. The occurrence and permit stores retain one-use consumption across subsequent attempts.

The outcome classification and stored disposition describe different facts. `EXACT`, `DIVERGED`, and `INDETERMINATE` classify outcome evidence under the profile. The journal records whether execution committed, was rejected before commitment, or requires resolution after possible execution. For example, the invoice candidate is divergent, but its saved disposition is rejection before commit.

S2. OBSERVATION BOUNDARY AND BACKEND REALIZATION

A. What the PostgreSQL Deployment Registers

The PostgreSQL adapter operates over an explicitly provisioned configuration. Registration specifies the business relations, supported operation entry points or DML, trigger and function definitions, execution role, audit objects, and deployment restrictions.

The public three-task path snapshots 11–13 registered relations, depending on the task. This includes relations that a task does not request to change. It does not discover its observation scope from the rows mentioned in the current approval. A trusted metadata connection reads the live catalog. The execution connection retains restricted privileges; administrative metadata access does not become an agent capability.

TABLE S2

COMPONENT-LEVEL BOUNDARY INVENTORY. EACH CONSTRUCTION HAS FIVE REPETITIONS; COUNTS REPORT SATISFIED CASE ASSERTIONS.

Configuration class	Cases	Pass	Refuse
Base / foreign database	4	10	10
Host / network	4	10	10
Asynchronous signals	5	10	15
Function / configuration drift	4	5	15
Continuation authority	4	5	15
Total	21	40	65

Boundary checks classify supported routes as observed or denied and refuse an unclassified route. Relevant inputs include relation and trigger definitions, callable functions and their permissions, role membership, database `TEMP` privileges, extension and foreign-data exposure, and the configured host/network restrictions. Application functions are admitted only within the supported registered configuration; this is not an unrestricted analysis of arbitrary function code.

Write-recording triggers attach stage identity to changes in the registered relations. The adapter forces deferred constraint triggers using `SET CONSTRAINTS ALL IMMEDIATE` before collecting evidence. It locks the registered relations before reading the initial state and keeps those locks until the held transaction commits or rolls back. Candidate-state and audit checks then concern the same transaction selected for promotion. Administrative grants, function implementations, and external isolation are deployment-managed configuration and must remain stable during protected execution. The final gate obtains fresh metadata and rejects visible changes relative to the certified configuration. This recheck supplements the relation locks: the locks stabilize compared data, while fresh certification detects configuration changes visible at the gate.

Snapshot cost over the 11–13 registered relations is part of this prototype’s observation work; the separate in-memory comparison benchmark measures only comparison computation.

B. Boundary Test Inventory

The component matrix contains 21 constructions, each repeated five times. Table S2 reports the tested configuration classes. Its counts are satisfied expected-pass and expected-refusal assertions from admission, certificate, and continuation components. Represented host, network, foreign-database, and notification capabilities are examined without invoking those outlets. Harness-initialized safety counters are not used as measurements of independently monitored outlet activity.

The base/foreign class tests a stable certificate, a denied foreign outlet, a foreign table, and a callable foreign-database path. Host/network cases pair denied capabilities with an exposed host role or unsupported host-facing extension. Signal cases include a benign statement, a registered benign function, and direct or wrapped notification paths. Function cases change public-execute permission, search path, or execute ACL relative to stable configuration. Continuation cases

compare a registered continuation with an independent reader, wrong authority lineage, and a replay.

A separate integrated study connects certification to actual held transactions and independent durable-state readback. It uses five scenarios with 20 trials each: a clean five-effect operation, an extra sibling outbox write, an exposed host capability before preparation, a host-capability grant after preparation, and reuse of a consumed permit. All 100 trials match their runtime and durable-state expectations. Clean operations commit; the extra-write and certification failures preserve initial application rows; replay leaves only the first approved result. The replay scenario contains two execute attempts per trial. Host-capability grants are inspected as catalog changes, without executing a host-file primitive.

C. What Each Backend Implementation Establishes

The implementations expose different control points:

- *PostgreSQL*: private native transaction, registered relation snapshots and protected audit, boundary certification, final rechecking, and one-use promotion.
- *STATE and ToolSandbox*: native business tools produce a private simulator snapshot. The held runtime checks that snapshot before selecting durable state in SQLite. These adapters cover simulator business state.
- *MySQL*: a database-side audit procedure compares predeclared row transitions before committing a native transaction. This is the evaluated MySQL adapter prototype.
- *GitHub*: a fresh GET checks registered issue fields after the modifying request. A saved local decision controls the continuation acknowledgement. It neither rolls back remote changes nor establishes attribution of all effects to that request.

The GitHub fields are title, body, state, labels, and assignees. A matching readback, an out-of-band title change, and unavailable readback are distinct study conditions. Historical classifier strings containing “exact” or “diverged” in these records denote field-match categories for this runner. The reported evidence is registered-field readback and local continuation control, as in the main paper’s RQ5.

S3. EXPERIMENTAL POPULATIONS, COMPARISONS, AND ADDITIONAL RESULTS

A. Public Tasks and Selection

MCPMark supplies the database tasks, STATE-Bench supplies shopping, customer-support, and travel tasks, and ToolSandbox supplies four single-action scenarios. Their public repositories are listed below.¹

The twelve independently approved STATE tasks in Table S3 are a purposeful, request-resolvable subset with four tasks per domain. Requests with unresolved choices or missing subsequent consent are not used to construct these specifications. Inputs include the released user messages preceding the first mutating invocation and the initial business data.

Native confirm=false previews are not counted as mutations in this independent subset. Approval generation is hand-authored application integration, not an evaluated language-model extraction task.

The broader STATE comparison retains all 206 historically eligible write-task candidates from the frozen inventory. A new native inventory pass covered 300 tasks with three replays each; it did not redefine the 206-task cohort using the new comparative outcomes. Nine tasks in that cohort perform writes but finish with no net state change. Their empty change targets remain in the population. Omitting or duplicating a call can also leave the final result unchanged, so injected variants are retained whether or not they cause a mismatch.

The three independently approved PostgreSQL tasks are *customer_data_migration*, *employee_hierarchy_management*, and *sales_and_music_charts*. Their specifications calculate customer rows, employee/report/customer assignments and salaries, and monthly invoice aggregates and music rankings, respectively.

Schema initialization is separately approved setup outside the held DML transaction. For the employee task, the detailed add-two/delete-one instructions and public verification query imply nine employees, while a final sentence says ten; the experiment explicitly adopts the detailed instructions. Ranking ties use source-name collation order read before execution, with quantities and revenues independently calculated.

The remaining eighteen tasks in the 21-task, 269-SQL-call MCPMark inventory have explicit implementation dispositions: ten need setup, namespace, sequencing, or business-contract support; five request functions, triggers, or privilege policies as outputs; one concerns physical index design; one concerns permission-catalog auditing; and one needs a vector-extension profile. These are unimplemented task/profile combinations, not measured successful executions or runtime rejections.

B. Conditions, Fault Placement, and Denominators

Direct PostgreSQL execution already uses a transaction and aborts on SQL errors. The AgentSpec condition uses the original Rule, RuleState, RuleInterpreter, Action, and Stop implementation from AgentSpec 0.1.0, with a custom deterministic SQL target predicate registered through its predicate11 slot. The rule checks visible write operations and target relations against the application’s relation-operation policy. It rejects the separate explicit-extra-write control in all nine runs. Hidden-write fixtures leave submitted SQL unchanged and modify the resulting state through backend triggers. The comparison concerns this call-level rule configuration.

For STATE, the error-reject baseline uses the same native handlers in a private copy and discards that copy if a tool raises, returns a nonempty error field, or reports success=false. The integrated condition compares the candidate outcome with approval before saving. Its effect-comparison ablation retains staging and permit handling but removes this comparison. Tool response errors and outcome mismatches are measured separately.

PostgreSQL fault fixtures add a Genre write, omit a requested row, or alter a requested value. STATE omission and

¹MCPMark: <https://github.com/eval-sys/mcpmark>; STATE-Bench: <https://github.com/microsoft/STATE-Bench>; ToolSandbox: <https://github.com/apple/ToolSandbox>.

TABLE S3

THE TWELVE INDEPENDENTLY SPECIFIED STATE TASKS. AN ASTERISK IDENTIFIES THE SIX LIVE-AGENT TASKS. TASK IDS ARE LOCAL TO THE NAMED DOMAIN.

Domain	Released task ID	Approved operation
Shopping	10-loyalty_points_on_discount	Add laptop with SAVE10 discount
	14-price_drop_disclosure	Add the specified laptop
	20-triple_policy_compound_disclosure*	Add two products; gift-wrap only the speaker
	84-remove_qty_scope_all_vs_one*	Remove the entire webcam cart line
Support	11-cancel_before_shipment*	Cancel an unshipped order
	14-cancel_split_payment	Cancel an order with split payment
	150-spare_cancel_in_transit_fee_decline*	Cancel only the unshipped item
	17-cancel_price_match	Refund the price difference
Travel	2-cancel_business_international*	Cancel two flights and the associated hotel
	100-challenge_medical_yyze_intl_lt7d	Change flight using the medical-fee rule
	103-change_business_international_fare_diff_only	Change international business-class flight
	104-change_domestic_gt7d_personal_fee_tier*	Change domestic flight; preserve preferences

TABLE S4

FULL 206-TASK COMPARISON. EACH COLUMN HAS 618 RUNS PER CONDITION. CLEAN COUNTS MATCHING DURABLE STATES; FAULT COLUMNS COUNT MISMATCHING COMMITS.

Condition	Clean ↑	Omit ↓	Duplicate ↓
Direct	618	354	138
Error-reject	597	138	138
EffectMatch	618	0	0
No result comparison	618	354	138

Matching durable state is distinct from issuing a commit. Error-reject saves 591 clean candidates; six other clean runs retain an initial state already equal to the reference, giving 597 matching states.

duplication modify the first selected write invocation. For the independently approved STATE subset, the two additional faults are candidate-snapshot injections after native replay: add one to the relevant cart total, refund amount, or booking price; or change an account email. The adapter and policy do not receive the fault label.

The twelve-task experiment has 20 effect cells and three additional binding cells per task, each repeated three times, yielding 828 executions. PostgreSQL has 16 effect cells and six binding/boundary cells per task, also repeated three times, yielding 198 executions. The 206-task comparison has three variants, four conditions, and three repetitions per task: 7,416 executions. Table S4 includes the ablated condition omitted from the main comparison table. Of 618 omission runs, 354 produce mismatching candidates; of 618 duplication runs, 138 do so. The remaining injected runs are still counted. Repetitions use fresh execution state, but repeated deterministic calls are not additional task identities or independent samples of agent behavior.

C. Full Mechanism Comparison

Table S5 expands the four scenarios shown in the main paper to all eight frozen scenarios. Each of the six conditions has 20 fresh runs per scenario, totaling 960 executions on local PostgreSQL, SQLite, and loopback HTTP fixtures. Direct and Allowlist are separate conditions. Robust Workflow adds constrained requests, current-state checks, readback, and

transaction/retry controls. NativeAudit adds ordinary backend audit evidence. Complete Audit is an instrumented observation oracle with the full protected-effect set and a locally derived expected set. EffectMatch additionally binds the approved source and execution context.

The oracle’s observation advantage explains its effect-level results. In the wrong-source scenario, the effects still match the locally supplied target, so observing them more completely cannot establish that the target came from the designated approver. Conversely, once a remote effect has occurred, a later blocked continuation does not undo it. The matrix keeps these outcomes separate from pre-commit rejection.

D. Ablation Interventions

The six dedicated fault pairs in Table S6 each contain 20 full and 20 ablated executions. Candidate binding and fresh-certificate capture also have 20 clean controls per condition. These are 240 fault executions plus 80 clean controls. Public-task comparison and binding ablations use their own task populations and three repetitions, as specified above.

Candidate-binding removal disables the runtime’s stage-identity checks and the adapter’s permit-to-candidate tuple check. Signature, expiry, current policy, base-state validation, candidate-to-own-observation validation, and one-use consumption remain active. The fixture first validates a file-backed invoice candidate with status paid, then presents its permit for a different candidate with status cancelled. Independent file readback identifies the committed candidate.

Fresh-boundary removal returns the genuine initial certificate at the final gate instead of recapturing metadata. The fault grants database TEMP permission after staging. The retained relational catalog check does not inventory that database-level privilege; full outlet recapture does. Both the catalog and business-effect checks remain active. The ablated result therefore still matches the approved five-effect deletion, but commits under a changed certification condition. No temporary table or host operation is invoked.

An earlier role-grant diagnostic was detected by both the outlet and catalog checks; it is retained separately and is not pooled into this discriminating TEMP-grant study. The

TABLE S5

ALL EIGHT MECHANISM SCENARIOS. EVERY DISPLAYED OUTCOME OCCURRED IN 20/20 RUNS IN ITS CELL. DIRECT AND ALLOWLIST WERE EXECUTED SEPARATELY. COMPLETE AUDIT IS THE INSTRUMENTED OBSERVATION ORACLE.

Scenario	Direct	Allowlist	Robust Workflow	NativeAudit	Complete Audit	EffectMatch
Clean chain	Exact	Exact	Exact	Exact	Exact	Exact
Extra PostgreSQL write	Extra commit	Extra commit	Extra commit	Reject	Reject	Reject
Mediated hidden field	Extra commit	Extra commit	Exact	Exact	Exact	Exact
Stale selector	Stale commit	Stale commit	Reject	Reject	Reject	Reject
Opaque remote extra	Extra; advance	Extra; advance	Extra; advance	Uncertain; stop	Diverged; stop	Diverged; stop
Readback unavailable	Response; advance	Response; advance	Reconcile	Uncertain; stop	Uncertain; stop	Uncertain; stop
Response loss / restart	Resend; duplicate	Resend; duplicate	Reconcile	Reconcile	Reconcile	Uncertain; stop
Wrong-source target	Accept local match	Accept local match	Accept local match	Accept local match	Accept local match	Reject

Exact: approved result is realized. Reject: no candidate promotion. Reconcile and uncertain: no dependent release or automatic resend. “Extra; advance” leaves one unapproved remote effect and runs one dependent action; “diverged; stop” and “uncertain; stop” in that row leave the same remote effect but block continuation. Direct/Allowlist resend once in the response-loss row, causing one extra effect. Wrong-source matching preserves the locally expected effect but lacks the designated approval source. These labels summarize recorded behavior; they do not introduce new protocol states.

TABLE S6

DEDICATED ABLATIONS: REMOVED MECHANISM AND THE FAILURE OBSERVED IN ALL 20 ABLATED FAULT RUNS. FULL EXECUTION REJECTS OR BLOCKS ALL 20 PAIRED FAULTS.

Removal	Discriminating intervention / outcome
Complete observation	Omit the additional write from <i>E</i> ; extra effect commits.
Approval-source check	Substitute the source while effects match; wrong-source approval accepted.
One-use execution binding	Reuse authority; an additional external effect occurs.
Durable hazard record	Lose saved divergence; a dependent action executes.
Candidate binding	Present a paid-candidate permit for a cancelled candidate; replacement commits.
Fresh boundary capture	Reuse the initial certificate after a TEMP grant; stale-boundary execution commits.

public-task occurrence ablations remove occurrence/action and pre-state binding together. Their tested intervention changes occurrence while leaving the candidate business state correct. The public PostgreSQL boundary ablation removes the combined final boundary/audit recheck; the dedicated TEMP study isolates fresh certificate capture. These interventions have different scopes and are reported separately in the main paper.

E. Recovery, Task-Level Timing, and Other Backends

Recovery and timing use the same eighteen STATE tasks: the six lexicographically first eligible multistep tasks per domain. Table S7 identifies every task and gives the execution medians underlying the main cost figure.

Recovery launches one producer and two fresh consumers per case, with six normal/interrupted scenarios and five repetitions. Producers exit at the selected checkpoint; consumers read saved state and terminal records without reconstructing missing terminals. A unique harness acknowledgement rep-

resents release of dependent work. Thus the correct-state/no-terminal cases measure withheld continuation pending reconciliation.

STATE timing includes native replay, runtime construction, comparison, permit handling, and durable SQLite selection. Input loading, clean-reference generation, initial store setup, resets, scoring, and diagnostic replay are outside the timer; two warmups per task and condition are excluded. All 1,080 timed executions produce correct results. The median across task-level held/direct ratios is 4.345, while the median added time across task medians is 1.081 ms. The two statistics answer different questions and are computed from unrounded data.

PostgreSQL certification timing uses 30 alternating pairs of one five-effect operation, with two warmup pairs excluded. Both conditions retain relational catalog checks, effect comparison, and one-use permits. Initial and final outlet capture each opens a fresh metadata connection. The disabled/enabled total medians are 13.292/29.127 ms; nearest-rank p95 values are 17.810/40.472 ms. The median paired increment is 15.157 ms. Fixture setup, fault installation, independent readback, and cleanup are outside this timer.

The separate 1–500-effect comparison measurement uses five trials of 1,000 comparisons per size and excludes staging and persistence.

MySQL’s eight authored operations are account activation, bulk transition, balance transfer, order archival, inventory upsert, zero-balance deletion, account archival projection, and joined bonus update. Each runs clean, extra-write, and omitted-statement variants in direct and audit-procedure conditions, with five repetitions: 240 executions.

ToolSandbox uses cellular_off, wifi_off, remove_contact_with_id, and update_contact_with_id_and_phone_number. Clean, extra-reminder, and omitted-action variants each have direct and held conditions with three repetitions: 72 executions. ToolSandbox approval uses the user request and initial state; milestone scoring runs after the decision.

GitHub’s eight shapes cover creation, title revision, body re-

TABLE S7

TASK-LEVEL MEDIANS FOR THE EIGHTEEN-TASK TIMING AND RECOVERY COHORT. EACH TASK HAS 30 ALTERNATING DIRECT/HELD TIMING PAIRS. TIMES ARE MILLISECONDS; THE RATIO DIVIDES THE TWO TASK MEDIANS.

Domain	Released task ID	Direct	Held	Ratio
Support	10-return_full_order	.309	2.281	7.37
	100-challenge_warranty_maxed_return_option	.287	1.507	5.25
	105-hard_price_match_after_exchange_request	.256	.979	3.82
	107-hard_bundle_price_match_requires_approval	.278	1.157	4.16
	11-cancel_before_shipment	.234	1.122	4.80
	111-hard_exchange_downgrade_cash_demand	.289	2.344	8.11
Shopping	10-loyalty_points_on_discount	.489	1.675	3.42
	100-goodwill_exception_fabrication_decline	.494	1.713	3.47
	111-hard_shipping_gold_express_vs_nextday	.257	1.412	5.50
	118-hard_loyalty_redemption_cap_preview	.259	1.545	5.96
	12-gift_wrap_silent_fee	.687	2.985	4.34
	138-hard_remove_desk_keep_invalid_promo	.282	1.595	5.66
Travel	100-challenge_medical_yyze_intl_lt7d	.303	1.309	4.31
	103-change_business_international_fare_diff_only	.308	1.281	4.16
	104-change_domestic_gt7d_personal_fee_tier	.286	1.235	4.32
	106-change_domestic_weather_zero_fee	.310	1.286	4.16
	109-change_international_gt7d_bereavement_discount	.287	1.246	4.35
	110-change_domestic_schedule_change_zero_fee	.282	1.250	4.43

vision, closing, reopening, title-plus-body revision, body-plus-closing, and reopening-plus-body revision. Each has five repetitions under clean, out-of-band title-change, and unavailable-readback conditions: 120 executions. The study saves intended fields before the write and obtains observed fields through a fresh GET. Only matching readback releases the local acknowledgement; the other conditions block it without resending. These are operation cases on one object type, rather than eight public benchmark tasks.

F. Execution Environments

The public-task experiments used an Apple M5 host. Native simulator clients used Python 3.12.13. The integrated boundary studies used PostgreSQL 17.6 on aarch64 with psychopg 3.3.4 and a Docker image based on pgvector:0.8.0-pg17-bookworm; the image digest is retained in the experiment manifest. MySQL used a disposable 8.4 instance. Database fixtures ran on local internal Docker networks.

The boundary experiments inspected privileged capabilities through controlled metadata changes rather than invoking host or external-network operations. Model inference is confined to the live-agent and separately reported surface-approver studies; deterministic replays, recovery, certification, and timing require no model calls or GPU computation.

S4. LIVE-AGENT INPUTS, CONFIGURATION, AND FINALIZATION

A. Shared Task Interface

The six starred tasks in Table S3 provide two operations per domain. Each is run with three configurations, two execution conditions, and three fresh repetitions, producing 108 episodes. Support task 11-cancel_before_shipment was also used in the disclosed pilot; pilot results are excluded. The formal schedule is shuffled with seed 20260921 and permits up to three isolated

episodes concurrently. Direct and integrated episodes generate their own call sequences.

Each configuration receives the native domain system prompt, formatted with task time and user identity; released user messages before the original trajectory’s first mutation; and native business-tool schemas. Two additional tools request the fixed experimental customer’s confirmation and finalize the task. The shared wrapper is:

Carry out the following customer request using only the provided business tools. The following are customer messages already received, in order; earlier confirmations apply only to their stated scope. Use the native preview/confirmation semantics. When customer confirmation is needed after preview, call request_customer_confirmation to obtain the reply, then continue. Call finish_task before giving your final response. Do not inspect files, run shell commands, use other tools, or consult external sources.

The message bundle follows this wrapper. No prior assistant/tool-result trace, expected snapshot, hidden target requirements, or scorer output is included. Some released travel messages already specify approved amounts; those amounts remain part of the public input. The approval specification is compiled and saved before inference, separately from the model-facing packet. Confirmation replies are fixed per task before execution and are released when the required native preview and target arguments are present. For the partial cancellation example, the reply approves cancelling only the unshipped phone case, reversing its payment to the original methods, and preserving the delivered phone. Reply eligibility uses preview metadata and arguments, not the expected state or benchmark score. This supplies a repeatable task interaction while preserving the stated scope of the customer’s request.

B. Models, Harnesses, and Budgets

The minimal DeepSeek condition uses native function calls through the API. DeepSeek+DSH uses DSH 0.1.5-rc.2 with

TABLE S8
ORIGINAL LIVE-AGENT COHORT. EACH ROW CONTAINS 18 EPISODES;
CANDIDATE AND DURABLE COLUMNS COUNT MATCHING BUSINESS
STATES.

Configuration	Path	Candidate	Durable	Finalized
DeepSeek API	Direct	18	18	15
	Integrated	18	15	15
DeepSeek+DSH	Direct	18	18	18
	Integrated	18	18	18
Codex	Direct	18	18	18
	Integrated	18	18	18

its SDK harness and native tools. Both request deepseek-flash, disable thinking, and cap output at 2,048 tokens per request. Codex CLI 0.154.0 requests gpt-5.6-sol with high reasoning. It runs in an ephemeral task workspace with only the episode’s business-tool server enabled; shell, web, personal skills, memories, and delegation are disabled. Model identifiers are the requested identifiers recorded at execution time.

Both DeepSeek configurations have a 24-model-iteration cap. All configurations have a 40-native-business-call and 180-second episode bound; Codex does not have a verified per-inference iteration cap. Harness-specific instructions and role/request formatting remain in place, so these are three agent configurations rather than an isolated comparison of model capability.

The direct path saves native state after each call. The integrated path keeps one private task candidate until finalization, when the held runtime checks the whole task result against independent approval before saving it.

C. Business-State Equivalence

The comparison rule is fixed before formal outcomes. Existing object identifiers, quantities, prices, preferences, unrelated state, and item multiplicities remain checked. Newly allocated shopping cart-item IDs may differ if a one-to-one mapping preserves their business fields and all associated references. Cart item-list order is ignored. This handles system-assigned surrogate IDs without accepting different products, quantities, or wrapping choices.

Strict raw-snapshot equality is recorded separately. One Codex/direct shopping episode differs only under this permitted representation rule; it is business-equivalent to the independent target.

D. Original Cohort and Targeted Follow-Up

Table S8 distinguishes candidate correctness, durable correctness, and finalization in the original 108 episodes. All candidate business states match their independent targets, and all episodes finish without process errors or timeouts. The batch therefore measures successful-task integration; controlled fault studies supply the incorrect-result prevention evidence.

The three unfinished integrated results are repetitions of the selective-gift-wrap shopping task. Their correct candidates remained private because the API agent omitted finish_task.

The host-finalization follow-up reruns exactly those three episodes with unchanged public packets, independent approvals, model settings, and budgets. On normal completed turns, the host invokes the existing checker if the model has not finalized; errors, timeouts, interruptions, and exhausted budgets do not trigger this hook. All three fresh candidates and durable states are correct. The main paper’s marked API cell combines the 15 original successes with these three targeted successes; the original cohort remains recorded separately here.

The original run made 645 native business-tool calls and 77 confirmation requests. Provider-reported usage was 5,457,463 input tokens, including cached input, and 80,697 output tokens; the three follow-ups used 45,191 input and 1,492 output tokens. These counts describe model/harness usage, not the runtime overhead measured in Section S3.

Per-episode records preserve public inputs, precomputed approvals, model events, native calls, candidate and durable snapshots, and available finalization receipts.